



СТРАТЕГИЯ · КОНТЕНТ · АВТОМАТИЗАЦИЯ

ContentOS для Codex и Claude Code

Пошаговое руководство: от папки с исходниками до готовых Reels, постов и каруселей, публикации и квалифицированной заявки.

V2.0 · ОБНОВЛЕНО 22 АВГУСТА 2026

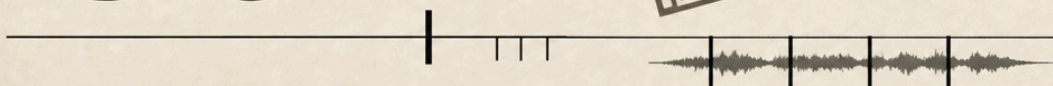
40 РАЗДЕЛОВ + РАБОЧИЙ СТАРТОВЫЙ КОД

ПРАКТИЧЕСКОЕ РУКОВОДСТВО

01

Вам не придётся изучать пятнадцать сервисов. Вы соберёте один центр управления, где Codex/Claude Code анализирует материалы, предлагает идеи, монтирует и передаёт работу дальше.

Content OS



ПРЕЖДЕ ЧЕМ ЧИТАТЬ ДАЛЬШЕ

Первый Reel за 15 минут

В комплекте с гайдом идёт `Content05-starter` — рабочий код, а не только описание: пять фото и один аудиотрек, автоматически нарезанные и склеенные в такт биты без единой ручной правки в видеоредакторе. Прежде чем читать 40 разделов архитектуры, получите такой же результат сами — это займёт 15 минут и не требует ни Codex, ни Claude Code:

1. Распакуйте `Content05-starter.zip`, откройте папку в терминале и выполните `npm install`.
2. Замените 3–5 фото и музыкальный трек в `examples/studio-clean-demo/photo` и `examples/studio-clean-demo/music` на свои.
3. Выполните `npm run demo:studio-clean` — это по очереди прогонит `ingest`, `analyze` (в том числе реальное определение битов трека), автоматическую нарезку фото под биты и `qa`. Если хотите видеть каждый шаг отдельно — команды разложены в разделе 27.
4. Откройте `examples/studio-clean-demo/output/reel.mp4`. Это именно то видео выше, только из ваших материалов.

Если на этом шаге всё получилось — дальше идёт объяснение, как это устроено и как расширить систему под ваш бренд и вашего агента. Если что-то не запустилось — раздел 29 (установка зависимостей) и README внутри архива покрывают типичные ошибки по шагам.

01

I. Ваша цель и итоговая система

Вместо набора разрозненных нейросетей вы получите одну понятную систему, в которой вы:

1. складываете фото, видео, музыку и короткие вводные в папку либо разрешаете агенту забирать их из Apple Notes и Apple Photos;
2. запускаете Codex/Claude Code одной командой или пишете задачу обычным языком;
3. он сам разбирает материалы, предлагает концепцию, хук и структуру;
4. сам создаёт монтажный план;
5. сам монтирует Reel через FFmpeg/Remotion;
6. сам создаёт субтитры, текстовые вставки, переходы, B-roll и варианты обложки;
7. из того же материала собирает пост и карусель в вашем голосе и вашей визуальной системе;

8. адаптирует готовый материал под Instagram, Telegram, MAX и VK;
9. после подтверждения ставит публикации в очередь;
10. ведёте человека из Reel в Direct-воронку ChatPlace и получаете уже квалифицированную заявку.

Центр системы — **Codex или Claude Code**. Он придумывает, принимает решения и управляет локальными инструментами. FFmpeg, Remotion, Whisper, HeyGen, ChatPlace и PostMyPost — исполнительные модули, которыми он управляет.

Как будет выглядеть работа:

Вы настроите рабочее место, где добавляете материалы и описываете задачу, а Codex/Claude Code сам превращает их в концепцию, сценарий, монтаж, тексты и готовые файлы. На старте вы подтверждаете идею и финальный ролик. Когда стиль стабилизируется, часть подтверждений можно убрать.

Важно: система не обещает «гарантированно вирусный Reel одной кнопкой». Она помогает быстро создавать и тестировать сильные варианты, но вирусность зависит от источников, идеи, упаковки, публикации и реакции аудитории.

И ещё важнее: это руководство — архитектура и пошаговый план сборки, а не готовая кнопка «нажал и получил Reel». Разделы 28–33 дают установку, каркас `AGENTS.md`, спецификации skills и техническое ТЗ на монтажный движок. Часть этого ТЗ уже реализована в стартовом коде `Content05-starter`, который прилагается к гайду (раздел 27) — он реально запускается и рендерит Reel из тестовой папки. Но под ваши материалы, бренд и визуальный стиль его придётся дорабатывать: это дни работы с Codex/Claude Code, а не пять минут чтения. Гайд рассчитан на читателя, который готов работать в терминале, ставить прм-зависимости и разбираться в ошибках рендера вместе с агентом — либо на того, кто закажет доводку до готового состояния отдельно.

2. Исходная точка и задачи

Текущая ситуация

- Это руководство не привязано к одному AI-провайдеру: логика — папка с исходниками, brand-контекст, `edit_plan.json`, `skills` — написана сразу под два инструмента, **OpenAI Codex** и **Claude Code**. Выбирайте любой из них или используйте оба параллельно на разных задачах — все команды и файлы в гайде даны для обоих. Единственная разница — на чём подписан ваш AI и как называется конфиг-файл (`AGENTS.md` у Codex, `CLAUDE.md` у Claude Code); сама архитектура одна.
- Если вам ближе Codex: он идёт в комплекте с подпиской ChatGPT, которая у многих читателей уже есть, — не нужно заводить отдельный аккаунт. Если вам ближе Claude Code: по опыту практиков он часто увереннее держит длинные многофайловые сессии редактирования кода — это субъективное наблюдение сообщества, а не измеренный бенчмарк, но для монтажного pipeline из раздела 8 это может быть заметно на практике. Требуется отдельного тарифа Claude (Pro, Max, Team, Enterprise или Console) — бесплатный план `claude.ai` доступа к Claude Code не даёт.
- Если у вас уже есть ChatGPT, на старте проверьте свой тариф, доступные лимиты Codex и вход под тем же аккаунтом. Если выбираете Claude Code — проверьте, что у вас Pro/Max/Team/Enterprise или настроен Console-биллинг.
- Codex: основной интерфейс — режим Codex в приложении ChatGPT Desktop, Codex CLI оставляем для диагностики и повторяемых команд. Claude Code: аналогично — вкладка «Code» в приложении Claude Desktop для работы без терминала, и CLI-команда `claude` для диагностики и повторяемых команд. Подробнее — раздел 13.
- Если ваш Instagram-профиль уже подключался к похожим сервисам (Virale, ManyChat, ChatPlace и подобным), перед настройкой новой воронки проверьте текущие подключения и доступы.
- У большинства читателей значительная часть контента сейчас собирается вручную, а задачи распределены между несколькими сервисами — это нормальная стартовая точка.
- Цель внедрения — получить повторяемый процесс, который можно запускать самостоятельно без постоянного заказа монтажа на стороне.
- Дальше вы будете работать с системой в чате: короткие команды, практика на своих материалах и сохранённые регламенты в `AGENTS.md`.

Ваши основные задачи

Ниже — типичный набор задач, с которым к такой системе приходят фотографы, мастера на заказ и небольшие бренды. Не все пункты обязательно относятся к вам — ориентируйтесь на свои приоритеты.

1. Продвинутый монтаж Reels из фото, видео и музыки.

2. Автоматический выбор сильных моментов, склейка под биты, переходы, хуки, мемные вставки и субтитры.
 3. Понимание разницы между обычным ChatGPT, режимом Codex и отдельным API-биллингом.
 4. Собственный цифровой аватар с единым фоном, одеждой и стилем.
 5. Разговорные ролики с суфлёром, субтитрами, примерами работ, блоками «до/после» и ценами.
 6. Автопубликация в Instagram, Telegram, MAX и VK.
 7. Аудит соцсетей, переупаковка, воронки и боты для заказов.
 8. Контент-система для второго профиля с другой аудиторией: портфолио, кейсы и профессиональное развитие для нужной вам категории людей — заказчиков, партнёров, работодателей.
-

03

3. Аккаунты и направления

Дальше в этом руководстве мы используем один сквозной пример: небольшая студия, которая превращает фото заказчика в готовое изделие — например, портрет на холсте. Если вы продаёте что-то другое, шаги остаются теми же — меняется только тип исходников и готового продукта. Ниже — типичная карта аккаунтов для такого бизнеса.

НАПРАВЛЕНИЕ	ПЛОЩАДКА	ЦЕЛЬ	ПРИОРИТЕТ
Основной бренд	Instagram @yourbrand	Заказы по фотографии	Главный коммерческий поток
Основной бренд	Telegram @yourbrand_channel	Доверие, кейсы, процесс, новости и продажи	Подключить к переиспользованию контента
Основной бренд	MAX	Дублирование ключевого контента и заявки	Подключить к публикационному контуру
Основной бренд	ВК, сообщество/канал	Контент и заявки внутри экосистемы ВК	Проверить фактический тип ресурса и права
Второй профиль (пример: портфолио для другого человека или ниши)	Instagram	Контракты, кастинги, показ работ — для профессии, где важно портфолио	Второй поток после запуска основной системы
Архивный/коллекционный канал (опционально)	Telegram	Наследие, архив, продажа отдельных работ коллекционерам	Отдельный контент-пакет, не смешивать с основным

Это может быть три-четыре разные аудитории. Нельзя вести все аккаунты одним стилем и одним универсальным промптом.

04

4. Результат первой недели

Что будет готово за семь рабочих дней

- Codex/Claude Code установлен и работает на вашем компьютере.
- Создан отдельный локальный workspace для контента.
- Созданы два бренд-контекста: [studio](#) и [second](#).
- Система принимает папку с материалами и составляет индекс исходников.

- Работает транскрибация речи и извлечение кадров из видео.
- Агент создаёт концепцию, три варианта хука и `edit_plan.json`.
- Работает минимум один шаблон монтажа для динамического Reel из материалов.
- Работает минимум один шаблон разговорного Reel с субтитрами и вставками.
- Получены 2–3 готовых ролика, на которых зафиксирован визуальный стиль.
- Создана пошаговая инструкция для самостоятельной работы.
- Подготовлена и протестирована одна Instagram-воронка для `@yourbrand`.
- Настроен тестовый контур публикации минимум для Instagram, Telegram, MAX и ВК либо зафиксированы конкретные ограничения доступа.

Что станет следующим этапом развития

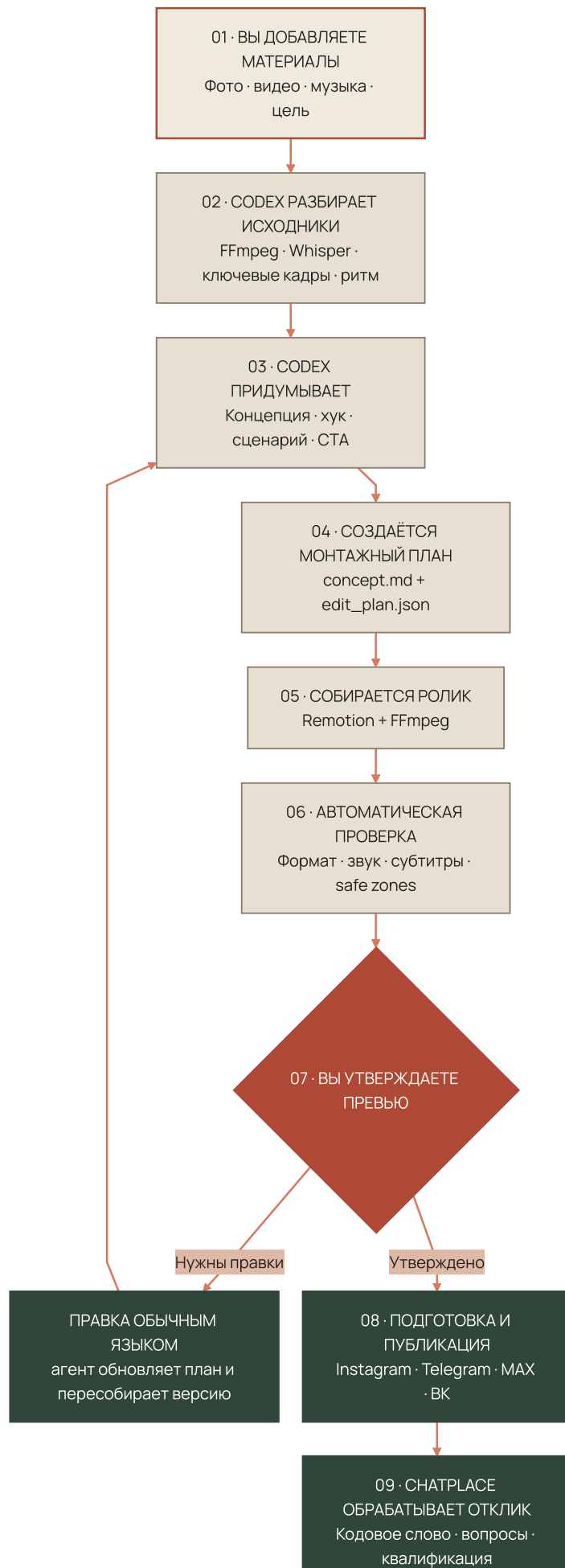
- идеально монтирующую систему для любого типа исходников;
- полностью автономного креативного директора без проверки человеком;
- десятки стилей монтажа;
- стабильный цифровой аватар во всех ракурсах и одежде;
- автопубликацию во все семь ресурсов без индивидуальной настройки прав;
- полноценную CRM, оплату, сложный Telegram/MAX-бот и сквозную аналитику;
- гарантированный рост охватов или продаж.

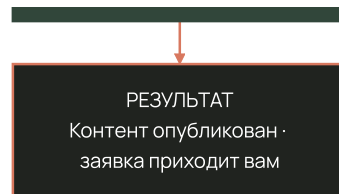
Первая неделя даст рабочую первую версию и навык самостоятельного использования. Дальше вы расширяете готовые шаблоны, а не переписываете систему с нуля.

05

5. Архитектура системы

Система устроена как один последовательный маршрут. Вы задаёте цель и добавляете материалы, а Codex/Claude Code управляет анализом, монтажом, проверкой, публикацией и заявочной воронкой.





Что здесь делает Codex/Claude Code

- читает брендовые инструкции и бизнес-цели;
- просматривает изображения и contact sheets;
- работает с транскриптами и таймкодами;
- выбирает сильные фрагменты;
- придумывает идею, хук, СТА и варианты подачи;
- создаёт строгий монтажный план;
- запускает локальные скрипты;
- рендерит и проверяет результат;
- вносит правки по обычной фразе пользователя;
- готовит варианты под каждую соцсеть;
- после разрешения обращается к API публикации и воронки.

Что Codex/Claude Code не делает «магически»

Агент не смотрит MP4 так же, как человек смотрит ролик на таймлайне. Поэтому видео сначала превращается в машинно-читаемый набор:

- ffprobe -метаданные;
- аудиодорожка;
- транскрипт с таймкодами;
- ключевые кадры;
- контактный лист;
- сцены и моменты смены кадра;
- пики громкости и биты;
- при необходимости — описание каждого фрагмента.

После этого агент принимает креативные решения и передаёт их детерминированному монтажному движку.

6. Технологическая основа

В качестве основы используется уже работающий прототип монтажного конвейера. Это позволяет начать с реальных роликов, а не тратить первую неделю на создание технического ядра с нуля.

Он уже умеет:

- принимать markdown-бриф;
- транскрибировать face-cam через Whisper;
- кешировать транскрипты;
- приводить мобильные видео к стабильному H.264/30 fps;
- рендерить вертикальное видео 1080×1920 через Remotion;
- делать karaoke-субтитры;
- накладывать хук;
- показывать face-cam полноэкранно или в split-layout со скринкастом;
- запускаться одной командой через локальный skill `reel`.

Перед внедрением прототип нужно очистить от старого брендинга и дополнить под ваши задачи. В первой версии нужно добавить:

- работу с набором из десятков фото и видео;
- автоматический отбор сцен;
- анализ музыки и beat-sync;
- `edit_plan.json` с произвольным таймлайном;
- несколько визуальных пресетов;
- B-roll, фото «до/после», цен и карточек продукта;
- брендовые профили для `studio` и `second`;
- генерацию трёх концепций;
- проверку финального ролика по кадрам;
- публикацию и аналитический feedback loop.

Правильный путь: сделать отдельную чистую версию проекта, а не менять настройки исходного прототипа глобально.

Часть этого списка не нужно строить с нуля. К гайду прилагается `Content05-starter` — рабочий референс-каркас, который уже умеет принимать job-папку, строить `manifest.json` через `ffprobe`, вытаскивать ключевые кадры, рендерить `edit_plan.json` в вертикальный mp4 через FFmpeg и проверять результат по

формату, длительности и звуку (раздел 27). Рендер сделан на чистом FFmpeg, а не на Remotion, специально: так старт работает на любой машине без headless-браузера и лишних мегабайтов зависимостей; переход на Remotion — это апгрейд для более сложной графики, а не обязательное условие для первого рабочего Reel. Это честный v0, а не готовая production-система: транскрипция речи, автоматический отбор сцен и визуальные пресеты в нём — заготовки, которые вы или агент дорабатываете под свой стиль и объём материала. Но для монтажа фото под музыку beat-sync в нём уже настоящий, а не симуляция: реальный анализ трека по амплитудной огибающей находит биты и темп, и автоматическая нарезка кадров под них запускается одной командой (раздел «Первый Reel за 15 минут» выше). Так первую неделю вы начинаете с рабочего ядра, а не с чистого листа.

07

7. Структура вашего рабочего пространства

```
content-os/
├── AGENTS.md
├── .env.example
├── .gitignore
├── .agents/
│   └── skills/
│       ├── content-director/SKILL.md
│       ├── reel/SKILL.md
│       ├── talking-head/SKILL.md
│       ├── repurpose/SKILL.md
│       ├── publish/SKILL.md
│       └── analytics/SKILL.md
├── brands/
│   ├── studio/
│   │   ├── brand.md
│   │   ├── offer.md
│   │   ├── prices.md
│   │   ├── faq.md
│   │   ├── visual.json
│   │   └── examples/
│   ├── second/
│   │   ├── positioning.md
│   │   ├── portfolio-facts.md
│   │   ├── visual.json
│   │   └── examples/
│   └── archive/
├── inbox/
│   └── 2026-08-example-job/
│       ├── request.md
│       ├── video/
│       ├── photo/
│       └── music/
└── work/
```

```
|   └─ 2026-08-example-job/
|       ├── manifest.json
|       ├── transcript.json
|       ├── frames/
|       ├── contact-sheet.jpg
|       ├── beats.json
|       ├── concept.md
|       └─ edit_plan.json
└─ templates/
    ├── montage/
    ├── talking-head/
    └─ portfolio/
└─ output/
    └─ 2026-08-example-job/
        ├── reel-v1.mp4
        ├── reel-v2.mp4
        ├── cover.jpg
        ├── captions.srt
        └─ platform-copy.md
└─ analytics/
    └─ content-results.csv
```

Пример request.md

Задача

Аккаунт: studio

Цель: получить заявки на ваш продукт по фотографии

Формат: динамический Reel

Длительность: 25–35 секунд

Исходники: использовать всё из текущей папки

Музыка: `music/main-track.mp3`

Обязательные элементы:

- показать исходную фотографию;
- показать 2–3 этапа работы;
- показать готовый результат крупно;
- добавить цену «от ...» только если она подтверждена в `prices.md`;
- СТА: написать слово «ЗАКАЗ».

Нельзя:

- придумывать сроки или цены;
- обещать результат, не подтверждённый в brand-файлах;
- использовать чужие работы;
- публиковать без подтверждения.

8. Источники материалов: Apple Notes и Apple Photos

Зачем это нужно

Сырьё для контента обычно уже лежит у вас на Mac. Идеи и черновики — в Apple Notes, куда вы накидываете мысли между делами. Видео и фото для монтажа Reels — в Apple Photos, куда всё сваливается автоматически с iPhone. Проблема не в том, что материала нет, а в том, что его приходится вручную перетаскивать из одного приложения в папку `inbox/` вашего проекта `content-os`.

Это ручное перетаскивание — самое узкое место конвейера. Пока вы сами открываете Notes, ищете нужную заметку, копируете текст, потом открываете Photos, ищете видео за нужный период и экспортируете его — агент простаивает. Задача этого раздела — научить Codex или Claude Code забирать материалы напрямую из обоих приложений, без вашего участия в этой рутине.

Важная оговорка сразу: с Notes всё работает предсказуемо через штатный AppleScript. С Photos штатный AppleScript почти полностью сломан, и рабочий путь там — отдельная утилита. Ниже подробно про обе части, с конкретными командами и ловушками, которые нашлись при проверке.

Apple Notes: чтение и экспорт через AppleScript

Notes управляется через `osascript` — встроенный интерпретатор AppleScript в macOS. Никаких дополнительных утилит ставить не нужно. Вот базовые запросы, которые агент может выполнять сам.

Список папок в Notes:

```
osascript -e 'tell application "Notes" to get name of every folder'
```

Команда отрабатывает мгновенно. Общее число заметок в приложении:

```
osascript -e 'tell application "Notes" to count of notes'
```

Поиск заметок по тексту тоже работает и делает это быстро — на проверке запрос уложился примерно в полсекунды:

```
osascript -e 'tell application "Notes" to get notes whose body contains "..."'
```

ЛОВУШКА

Если обратиться к папке по имени напрямую, а внутри неё есть подпапки, AppleScript вернёт 0 заметок — даже если содержимое там есть. Заметки лежат глубже, чем видит простой запрос к папке верхнего уровня. Чтобы получить весь контент, нужен рекурсивный обход: сначала список подпапок, потом заметки в каждой из них отдельно. Если агент говорит, что папка пустая, а вы точно знаете, что там что-то есть, — первым делом проверяйте структуру подпапок.

body заметки возвращает HTML-разметку, а не чистый текст и не markdown. Для дальнейшей работы с текстом — вставки в статью, передачи в промпт — HTML неудобен, его нужно конвертировать.

Рабочая связка для экспорта: выгрузить body заметки в файл через osascript, затем прогнать через pandoc. Pandoc уже стоит через homebrew (проверенный путь бинарника — /opt/homebrew/bin/pandoc). Пример конвертации:

```
pandoc -f html -t markdown note.html -o note.md
```

Списки и жирный текст при такой конвертации сохраняются корректно. На массовой выгрузке папки из 13 заметок с разделителями-маркерами между ними вся операция заняла 1.14 секунды — узким местом это точно не будет.

ЛОВУШКА

Прямой доступ к файлу базы данных Notes (~/Library/Group Containers/group.com.apple.notes/NoteStore.sqlite) не работает без разрешения Full Disk Access. На проверке это дало ошибку ls: Operation not permitted при попытке посмотреть файл и Error: unable to open database "...": authorization denied при попытке открыть его напрямую через sqlite. Для задач этого раздела SQLite не нужен вообще — всё решается через AppleScript. Не тратьте время на обход через базу данных.

Разрешения настраиваются один раз. Откройте System Settings → Privacy & Security → Automation и включите галочку «Notes» напротив терминала или приложения, через которое работает агент (Terminal, iTerm, VS Code и так далее — смотря откуда вы запускаете Codex или Claude Code). При самом первом обращении к

Notes из терминала macOS сама покажет системный диалог с формулировкой вида «... хочет получить доступ для управления "Notes"» — нужно нажать «ОК».

Apple Photos: почему AppleScript не подходит

С Photos интуитивный путь — тот же AppleScript, что и для Notes — не работает. Штатный словарь команд Photos сломан на базовых операциях, и это подтверждено конкретными ошибками, а не догадками.

Фильтр по дате падает с ошибкой конвертации типа:

```
-- media items whose date is greater than ...  
execution error: Не удается превратить date в тип specifier. (-1700)
```

Фильтр по типу медиа (например, только видео) падает с синтаксической ошибкой:

```
-- media items whose media kind is video  
syntax error: Предполагаемый результат — конец строки; полученный — свойство. (-2741)
```

Обращение к альбому по имени в принципе выполняется без ошибки, но возвращает 0 элементов — то есть молча не работает. Команда `export` формально работает, но на файл уходит около 14 секунд, и итоговые имена файлов превращаются в бессмысленные строки вида `438d7c1285f14d64a747a84461fcefab.mov` — по ним потом невозможно понять, что это за видео. Список альбомов через `osascript` тоже получить можно, но это занимает около 8 секунд, а имена альбомов иногда дублируются.

Вывод простой: для Notes AppleScript достаточно, а для Photos нужен отдельный инструмент.

osxphotos: рабочий путь для выгрузки медиа

`osxphotos` <<https://github.com/RhetTbull/osxphotos>> — консольная утилита, которая читает библиотеку Photos напрямую и решает все проблемы AppleScript: фильтры по дате и типу работают, имена файлов остаются осмысленными, альбомы находятся по имени.

Установка через `uv`:

```
uv tool install osxphotos --python 3.11
```

ЛОВУШКА

osxphotos требует Python 3.10 или новее. Если поставить его под системный python3 (в проверенной системе это была версия 3.9), инструмент падает при запуске с ошибкой `TypeError: unsupported operand type(s) for |: 'type' and 'NoneType'`. Явно указывайте версию питона при установке, как в команде выше — `--python 3.11`. На проверке встала версия osxphotos 0.76.1, бинарник оказался в `~/.local/bin/osxphotos`.

Первая команда для знакомства с библиотекой — общая сводка:

```
osxphotos info
```

Выполняется около 8.5 секунд. Дальше можно считать медиа по фильтрам, не экспортируя ничего:

```
osxphotos query --only-movies --count
```

На проверенной библиотеке это вернуло 717 видео за 12.7 секунды (первый запуск дольше — утилита строит индекс, повторные быстрее). С фильтром по дате:

```
osxphotos query --only-movies --from-date "2026-07-23" --count
```

Вернуло 94 видео за месяц, 7.5 секунды. Главный сценарий для конвейера — не просто посчитать, а выгрузить последние видео в папку проекта:

```
osxphotos export <папка> --only-movies --from-date "2026-07-23" --limit 5 -V
```

Оригинальные имена файлов при экспорте через osxphotos сохраняются (`IMG_6374.MOV` и подобные) — в отличие от AppleScript-экспорта с его бессмысленными хэшами.

ЛОВУШКА

Это самое важное предупреждение в разделе. В проверенной библиотеке 3876 из 4001 ассетов оказались помечены как `missing` — то есть не скачаны локально, а лежат только в iCloud. Из 94 видео за месяц реально выгрузилось только 5, остальные 73 были пропущены с сообщением «`Skipping missing original photo`». Инструмент при этом не падает и не сообщает об ошибке — он просто молча отдаёт меньше файлов, чем вы ожидали. Человек в этот момент обычно решает, что команда не сработала правильно, хотя на самом деле файлов физически нет на диске.

Решение: в приложении Photos включить «Загрузить оригиналы на этот Mac» в настройках iCloud, чтобы медиа хранилось локально, либо заранее вручную скачать нужные файлы. Перед выгрузкой большого объёма стоит проверить поле `ismissing` у нужных ассетов через `osxphotos query --json`, чтобы понимать реальный охват до запуска экспорта.

Другие полезные флаги команды `export`. Фильтр по конкретному альбому реально работает (проверено на непустом альбоме), в отличие от аналогичной попытки в AppleScript:

```
osxphotos export <папка> --album "Название альбома"
```

Раскладка по папкам дат:

```
osxphotos export <папка> --directory "{created.year}/{created.month}"
```

Флаг `--sidecar json` сохраняет метаданные каждого файла рядом в отдельном json. Нюанс: `sidecar`-файл создаётся и для `missing`-элементов тоже, поэтому `json`-файлов в итоге будет больше, чем реально скачанных медиа — не путайте одно с другим при подсчёте.

Повторный запуск с флагом `--update` пропускает уже выгруженные файлы (в логе будет что-то вроде `skipped: 5`) — это значит, что инкрементальную выгрузку можно запускать регулярно, не боясь задублировать материалы в `inbox/`.

Для отбора конкретных видео под задачу полезен запрос в `json`:

```
osxphotos query --only-movies --json
```

Он отдаёт около 99 полей на каждый ассет. Из них для отбора контента реально пригодятся:

ПОЛЕ	ЧТО СОДЕРЖИТ
<u>exif_info.duration</u>	длительность видео в секундах, например 17.507
<u>fps</u> , <u>codec</u>	технические параметры файла
<u>width</u> , <u>height</u>	разрешение
<u>date</u> , <u>date_original</u> , <u>date_added</u>	даты съёмки и добавления в библиотеку
<u>albums</u> , <u>keywords</u> , <u>favorite</u>	организация и метки внутри Photos
<u>ismissing</u>	лежит ли файл локально или только в iCloud
<u>path</u> , <u>filename</u> , <u>original_filename</u>	расположение и имя файла
<u>persons</u>	распознанные лица, если нужен отбор по людям в кадре

ВАЖНО

Full Disk Access для osxphotos на проверке не потребовался — ошибку доступа воспроизвести не удалось ни на одной из команд выше. Не утверждайте клиентам или себе, что это разрешение обязательно нужно заранее. Если при запуске всё же появится ошибка доступа, она решается через System Settings → Privacy & Security → Full Disk Access — добавьте туда терминал так же, как для Notes.

Как отдать это агенту

Голые команды в терминале — это шаг вперёд по сравнению с ручным перетаскиванием файлов, но цель конвейера в другом: агент должен вызывать их сам, без вашей команды на каждый чих. Для этого нужны две вещи — обёртки-скрипты в проекте и строки в файле инструкций, которые объясняют агенту, когда их запускать.

В папке scripts/ вашего проекта content-os заведите пару простых shell-скриптов, которые оборачивают проверенные команды: один вызывает osxphotos export с нужными флагами и кладёт результат прямо в inbox/ , второй выгружает и конвертирует заметки из Notes в markdown тем же способом, что описан выше. Скрипт — это просто фиксация рабочей команды под понятным именем, чтобы агент не собирал её заново каждый раз и не рисковал забыть флаг вроде --from-date .

Дальше в файле инструкций проекта (AGENTS.md для Codex или CLAUDE.md для Claude Code) опишите словами, когда и как это использовать. Пример формулировки:

Источники материалов

Видео и фото для монтажа лежат в Apple Photos, черновики и идеи — в Apple Notes.

Не проси пользователя перетаскивать файлы вручную — забирай материалы сам.

- Свежие видео за период: `scripts/fetch-photos.sh <дата-от> <лимит>`
Кладёт файлы в `inbox/`. Если видео меньше, чем ожидалось, — часть могла быть пропущена как `missing` (не скачана из iCloud), проверь через ``osxphotos query --only-movies --from-date <дата> --json`` поле `ismissing` и предупреди об этом пользователя, не выдавай неполную выборку за полную.
- Заметки по ключевому слову: `scripts/fetch-notes.sh "<запрос>"`
Ищет по `body` заметок и конвертирует найденное в `markdown` через `pandoc`. Если результат пустой, а пользователь уверен, что заметка есть — проверь, не лежит ли она в подпапке: список папок верхнего уровня не показывает вложенные.

После этого пользовательский запрос в чате с агентом может выглядеть просто:

Забери все видео из Photos за последнюю неделю и заметку "идея про воронку" из Notes, положи в `inbox` проекта `reels-августа`.

Агент сам вызовет оба скрипта, сам сообщит, если часть видео пропущена из-за iCloud, и сам разложит результат в нужную папку. Ручной работы с интерфейсом Notes и Photos в этом сценарии не остаётся вообще.

Чек-лист готовности

- ✓ в System Settings → Privacy & Security → Automation включён доступ приложения-терминала к Notes;
- ✓ `osxphotos` установлен через `uv` с явным указанием `--python 3.11` или новее, бинарник доступен из PATH;
- ✓ в Photos для нужных альбомов включена загрузка оригиналов на Mac, либо вы понимаете, что часть медиа будет пропущена как `missing`;
- ✓ в проекте есть `scripts/fetch-photos.sh` и `scripts/fetch-notes.sh`, оборачивающие проверенные команды;
- ✓ в AGENTS.md или CLAUDE.md прописано, какими скриптами и при каких формулировках пользователя агент должен пользоваться;

- ✓ проверен реальный запуск на своей библиотеке — количество выгруженных файлов совпадает с ожиданием или расхождение объяснено полем `ismissing`.
-

09

9. Монтажный конвейер

Шаг 1. Приём и индексирование исходников

Codex/Claude Code:

1. проверяет структуру входной папки;
2. считает длительность, разрешение, fps, кодек и ориентацию каждого файла;
3. определяет вертикальные и горизонтальные исходники;
4. отмечает дубли и битые файлы;
5. создаёт `manifest.json`;
6. при необходимости приводит копии к CFR 30 fps, не трогая оригиналы.

Шаг 2. Разбор содержания

- Whisper делает транскрипцию речи с таймкодами слов.
- FFmpeg извлекает ключевые кадры и contact sheets.
- Scene detection отмечает смены планов.
- Агент описывает содержание кадров и выделяет лучшие фрагменты.
- Для фото фиксируются сюжет, качество, крупность, ориентация и возможная роль: обложка, доказательство, процесс, «до/после», СТА.

Шаг 3. Анализ музыки

- Из трека извлекаются BPM, биты, дропы, тихие и активные секции.
- Создаётся `beats.json` с точными таймкодами.
- Агент выбирает ритм монтажа: не каждый бит обязан быть склейкой.
- Коммерческая версия использует оригинальную или royalty-free музыку.
- Если нужен трендовый трек Instagram, основной Reel экспортируется без защищённой музыки, а трек добавляется внутри Instagram вручную.

Шаг 4. Креативная концепция

Codex/Claude Code выдаёт три варианта:

1. самый понятный и коммерческий;
2. более эмоциональный/историйный;
3. более динамичный/экспериментальный.

Для каждого:

- хук первых 1–2 секунд;
- обещание зрителю;
- последовательность сцен;
- текст на экране;
- музыкальная логика;
- СТА;
- риск или ограничение исходников.

На первой неделе вы подтверждаете один вариант. После накопления статистики система может выбирать сама по правилам.

Шаг 5. edit_plan.json

Пример сокращённой структуры:

```
{
  "format": {"width": 1080, "height": 1920, "fps": 30},
  "brand": "studio",
  "durationSec": 29.5,
  "hook": "Она прислала обычное фото. Вот что получилось.",
  "music": {"file": "music/main-track.mp3", "startSec": 12.4},
  "timeline": [
    {
      "from": 0.0,
      "to": 1.8,
      "source": "photo/source.jpg",
      "effect": "slow-zoom",
      "text": "Обычное фото → готовое изделие"
    },
    {
      "from": 1.8,
      "to": 4.2,
      "source": "video/process-01.mp4",
      "sourceIn": 3.1,
      "transition": "beat-cut"
    }
  ]
}
```

```
  ],
  "captions": {"enabled": true, "preset": "studio-clean"},
  "cta": {"text": "Напишите «ЗАКАЗ» — рассчитаем стоимость", "from": 25.5}
}
```

JSON нужен не для красоты. Он отделяет решения агента от рендера и позволяет менять один элемент без пересборки всей логики.

Шаг 6. Рендер

- Remotion отвечает за композицию, типографику, анимацию, карточки цены, «до/после», safe zones и повторяемый дизайн.
- FFmpeg отвечает за перекодирование, нарезку, звук, нормализацию, склейки и технический экспорт.
- Для каждого бренда используются свои цвета, шрифты, расположение субтитров и СТА.
- Результат: H.264/AAC, 1080×1920, 30 fps, плюс обложка и .srt.

Шаг 7. Автопроверка

Перед показом вам агент должен:

- проверить, что файл открывается и имеет правильный формат;
- проверить длительность, разрешение, fps и аудиодорожку;
- сделать contact sheet финального ролика;
- найти чёрные/пустые кадры;
- проверить, что текст не выходит за safe zones;
- сверить субтитры с транскриптом;
- проверить, что цена и факты взяты из брендовых файлов;
- убедиться, что СТА присутствует;
- сохранить отчёт qa.md.

Шаг 8. Правки обычным языком

Примеры ваших команд:

```
Сделай первые три секунды быстрее, убери второй кадр процесса,
замени музыку на более спокойную и подними субтитры выше.
```

Оставь монтаж, но сделай ещё два варианта хука и разные обложки.

Агент обновляет `edit_plan.json`, пересобирает только нужные части и показывает новую версию.

10

10. Разговорные ролики и аватар

Живой talking-head — сначала

Первый стабильный формат:

1. Codex/Claude Code изучает тему и пишет короткий сценарий.
2. Создает версию для суфлёра: короткие строки, разговорный язык, пометки пауз.
3. Вы записываете вертикальное видео.
4. Pipeline удаляет длинные паузы, делает субтитры, добавляет примеры работ, цены и «до/после».
5. Агент создаёт 2–3 варианта хука из одной записи.

Цифровой двойник — после утверждения живого формата

Подходящие исполнительные сервисы:

- Captions AI Twin — цифровой двойник, смена фона/одежды, генерация из сценария;
- HeyGen Digital Twin — персональный аватар по обучающему видео, хорошо подходит для повторяемых роликов и API-контур.

Codex/Claude Code должен:

1. написать сценарий;
2. отправить его в API выбранного сервиса;
3. дождаться готового аватарного клипа;
4. скачать клип в `work/` ;
5. пропустить его через общий Remotion-пайплайн;
6. добавить реальные примеры работ, цены, B-roll и субтитры;

7. показать preview до публикации.

Нельзя строить весь контент только на аватаре. Для физического продукта критично показывать реальные материалы, руки, фактуру, процесс и живого автора.

11

II. Какие skills подключить

Skills для Codex и Claude Code

SKILL	ЗАДАЧА	ЧТО ПОЛУЧАЕТ НА ВХОДЕ	ЧТО ОТДАЁТ
<u>content-director</u>	Выбирает тему и подачу на основе бизнеса, источников и метрик	бренд, цель, папка источников	3 концепции и приоритет
<u>reel</u>	Полный цикл динамического Reel	<u>request.md</u> + media	<u>edit_plan.json</u> , mp4, cover, QA
<u>talking-head</u>	Сценарий, суфлёр, субтитры и B-roll	тема или запись	script, teleprompter, mp4
<u>repurpose</u>	Делает платформенные версии	утверждённый master-контент	Instagram/TG/MAX/VK тексты и файлы
<u>publish</u>	Ставит утверждённое в очередь	package + дата	черновики/очередь PostMyPost
<u>analytics</u>	Замыкает feedback loop	метрики публикаций	разбор и новые правила

Brand skills/контексты

Нужны минимум два независимых пакета:

- studio-brand : оффер, цены, сроки, FAQ, доставка, визуальный стиль, допустимые обещания, СТА;

- second-brand : позиционирование для того, кто принимает решение (заказчика, работодателя, агентства), подтверждённые достижения, портфолио, языки, география и контакты.

archive-brand создаётся отдельно, если у вас есть такой третий поток и найдётся время на него.

Базовые skills для адаптации

- reel — рабочий прототип рендера, но его нужно отвязать от исходного демо-бренда;
- content-strategy — логика рубрик, целей и контентных опор;
- social-content — адаптация контента и календарь;
- chatplace — создание и управление Instagram-воронками;
- postmypost — публикация через API;
- carousel-designer — позже, для каруселей с кейсами и «до/после».

Перед установкой skills нужно очистить от старых путей, брендинга и настроек других проектов. Все токены и API-ключи хранятся в .env, а не внутри SKILL.md.

chatplace, postmypost и монтажные skills — это проектные workflows, а не встроенные функции самого агента. Skill описывает последовательность действий; реальную работу с сервисом выполняют его API, MCP/plugin или локальный скрипт.

Как skills подключаются в Codex и Claude Code

Структура:

```
.agents/skills/reel/SKILL.md
.agents/skills/content-director/SKILL.md
```

Агент автоматически видит skills по их описанию. Для предсказуемой работы вызывайте их явно: `$reel`, `$content-director` и т. д. В Codex CLI и в Claude Code список можно открыть через /skills. AGENTS.md хранит постоянные правила проекта и автоматически загружается в каждой сессии.

12. Как вы будете пользоваться системой

Самый простой ежедневный сценарий

1. Создать новую папку в `inbox/`.
2. Скопировать туда фото, видео и музыку.
3. Заполнить короткий `request.md` или просто написать агенту задачу.
4. Открыть папку проекта в агенте: режим Codex в приложении ChatGPT Desktop или команда `codex` в корне проекта; для Claude Code — вкладка «Code» в приложении Claude или команда `c'laude` в той же папке.
5. Запустить:

```
$reel собери коммерческий Reel для studio из новой папки.  
Сначала покажи три концепции, не публикуй без моего подтверждения.
```

6. Выбрать концепцию.
7. Посмотреть готовый Reel.
8. Написать правки обычными словами.
9. После подтверждения запустить:

```
$repurpose подготовь версии для Instagram, Telegram, MAX и VK.
```

10. Затем:

```
$publish создай черновики на завтра, ничего не публикуй сразу.
```

Режим после стабилизации

Когда сделано 10–15 роликов и понятны рабочие паттерны:

- система сама выбирает стиль по типу источников;
- генерирует три хука;
- рендерит три коротких варианта;
- вы выбираете один;
- публикационные черновики создаются автоматически;
- аналитика возвращается в content-results.csv ;
- analytics обновляет рекомендации, но не переписывает брендовые правила без подтверждения.

13. Сервисы и роли

ИНСТРУМЕНТ	РОЛЬ	РЕШЕНИЕ НА СТАРТЕ
ChatGPT с режимом Codex	Главный оператор: креатив, файлы, скрипты и QA	Один из двух; вход через ChatGPT, тариф проверяем по доступным лимитам
Claude Code	Тот же главный оператор, альтернатива Codex	Один из двух; вход через подписку Claude, работает в той же папке проекта
FFmpeg/ffprobe	Нарезка, конвертация, кадры, звук, экспорт	Обязательно, локально
Remotion	Программный дизайн и повторяемый рендер	Обязательно
Whisper через Groq или локально	Субтитры и таймкоды	Обязательно
Python beat analysis	BPM, биты, пики, секции	Обязательно для музыкального монтажа
HeyGen или Captions	Только генерация цифрового двойника	После живого talking-head MVP
ChatPlace	Instagram Direct, комментарии, кодовые слова и квалификация лида	Подключить к вашему основному аккаунту
PostMyPost	Публикация и платформенные черновики	Тест 7 дней, затем решение по тарифу
CapCut/Captions editor	Ручной аварийный редактор	Не центр системы, только fallback

ИНСТРУМЕНТ	РОЛЬ	РЕШЕНИЕ НА СТАРТЕ
Обычный чат: ChatGPT Chat/Work или Claude в приложении	Исследования, документы и быстрые обсуждения вне монтажной папки	Тот же аккаунт; основной production-контур остаётся в агенте

14

14. ChatGPT Desktop и Codex CLI: два интерфейса

Оба поддерживаемых инструмента гайда — Codex и Claude Code — устроены одинаково: у каждого есть настольное приложение с чатом и отдельный CLI для терминала, и оба читают одну и ту же папку проекта и одни и те же skills (разница только в имени конфиг-файла и пути к папке skills — см. разделы 30–32).

Codex: ChatGPT Desktop + Codex CLI

Codex можно использовать двумя способами, но папка проекта, AGENTS.md и skills у них общие:

- **ChatGPT Desktop → Codex** — основной понятный интерфейс: открыть папку, написать задачу и видеть изменённые файлы и результат.
- **Codex CLI** — терминальный интерфейс для диагностики, автоматизации и повторяемых команд.
- В июле 2026 отдельный Codex app вошёл в приложение ChatGPT Desktop. Отдельного «Codex Desktop» искать не нужно.
- Вход через ChatGPT использует лимиты выбранного тарифа. Вход с API-ключом считается отдельно по тарифам OpenAI API.
- Проверка установки: codex --version, codex login status, codex doctor. Внутри CLI используем /status, /usage и /skills.

Вашим интерфейсом станет не терминал как таковой, а 4–6 понятных команд, одна папка inbox и режим Codex в ChatGPT Desktop.

Claude Code: Claude Desktop (вкладка «Code») + CLI

У Claude Code та же логика двух интерфейсов, что и у Codex:

- **Claude Desktop** → вкладка «Code» — приложение для macOS, Windows и Linux со встроенным Code-режимом: работаете с той же папкой проекта без открытия терминала, аналог Codex-режима в ChatGPT Desktop.
- **Claude Code CLI** — терминальный интерфейс, команда `claude`. Запускается прямо из папки проекта; при первом запуске откроется окно браузера для входа в аккаунт.
- Нужен платный тариф — Pro, Max, Team, Enterprise или Console (API-биллинг по факту использования). Бесплатный план `claude.ai` доступа к Claude Code не даёт.
- Проверка установки: `claude --version`, а для полной диагностики (состояние установки, валидация настроек) — `claude doctor`.

Так же, как и с Codex, вашим повседневным интерфейсом станет не голый терминал, а те же 4–6 понятных команд и одна папка `inbox` — просто через Claude Code вместо Codex.

15

15. ChatPlace: что именно настраивать

ChatPlace не нужен как монтажный сервис и не нужен для всех каналов сразу. Его первая задача — превратить просмотры вашего основного аккаунта в квалифицированные заявки.

Первая воронка

```
Reel → комментарий или Direct «ЗАКАЗ»  
→ первое сообщение с кнопкой  
→ короткое объяснение  
→ количество людей  
→ размер/формат  
→ фото  
→ срок  
→ город/доставка  
→ ориентир по бюджету  
→ контакт  
→ тег qualified  
→ уведомление вам  
→ ручное подтверждение цены и заказа
```

Почему первое сообщение с кнопкой: Instagram разрешает боту после комментария отправить только одно первое сообщение; нажатие кнопки продолжает диалоговое окно.

Теги

- source_reel_{id} ;
- order_interest ;
- photo_received ;
- qualified ;
- needs_human ;
- quote_sent ;
- ordered .

Что не делать сразу

- не подключать все Instagram и Telegram-аккаунты;
- не покупать AI Manager до появления проверенного FAQ и устойчивой ручной продажи;
- не пытаться автоматически называть финальную цену для нестандартного заказа;
- не оставлять воронку без ручного takeover;
- не запускать без теста со второго аккаунта.

Тарифная логика на 6 августа 2026 года

- Free: 1 аккаунт/бот, до 200 активных контактов в месяц;
- Pro: 2 000 ₽/месяц, 1 аккаунт, автоматизации без лимита контактов;
- дополнительный аккаунт при помесечной оплате: 2 000 ₽/месяц;
- Creator: 4 500 ₽/месяц, включает Virale и AI-функции.

На старте лучше запустить воронку для вашего основного аккаунта на Free или Pro. Не оплачивайте Creator только потому, что был пробный Virale. Решение принимайте после измерения реального использования.

16. ChatPlace в Codex и Claude Code: подключение MCP

В предыдущем разделе вы настроили первую воронку в кабинете ChatPlace: подписку, теги, тарифы. Дальше есть выбор. Можно управлять всем через веб-интерфейс, кликая по шагам мышкой. А можно подключить ChatPlace к агенту и дальше собирать воронки, отправлять рассылки и читать отчёты прямо из чата с Codex или Claude Code, не открывая браузер.

Это раздел про второй вариант. Он технический, но не требует навыков программирования — только один раз скопировать команду и ключ.

Что такое MCP

MCP — это протокол, по которому агент получает доступ к внешнему сервису как к набору инструментов. Проще говоря: сейчас Claude Code или Codex умеет читать и писать файлы у вас на компьютере. После подключения MCP-сервера ChatPlace агент точно так же начинает видеть команды ChatPlace — создать воронку, отправить рассылку, посмотреть отчёт — и может вызывать их сам, когда вы просите об этом обычными словами на русском.

Никакого отдельного языка запросов учить не нужно. Вы пишете «собери воронку под кодовое слово в комментариях», агент сам выбирает нужный инструмент ChatPlace и вызывает его.

Как подключить

ChatPlace предоставляет MCP-сервер по адресу <https://mcp.chatplace.io/mcp>. Подключение идёт по HTTP, авторизация — через ключ вида `срк_...`. Ключ выпускается в кабинете ChatPlace, в разделе «Настройки → API ключи». Программного способа получить ключ нет — только руками в вебе.

Для Claude Code выполните в терминале одну команду:

```
claude mcp add chatplace --transport http https://mcp.chatplace.io/mcp --header "Authorization: Bearer ВАШ_КЛЮЧ"
```

Для Codex CLI добавьте секцию в файл `~/.codex/config.toml`:

```
[mcp_servers.chatplace]
url = "https://mcp.chatplace.io/mcp"
bearer_token_env_var = "CHATPLACE_API_KEY"
```

И пропишите саму переменную окружения, например в ~/.zshrc :

```
export CHATPLACE_API_KEY="ВАШ_КЛЮЧ"
```

Синтаксис для Codex взят из документации и на живой установке не проверялся — если конфигурация не подхватится с первого раза, сверьтесь с актуальным форматом config.toml в справке Codex.

ВАЖНО

Claude Code подключает MCP-сервер в контексте конкретного проекта — то есть той папки, из которой вы запустили команду `claude mcp add`. Если запустить агента из другой папки, сервера ChatPlace там не будет, и придётся добавлять заново.

Проверить, что подключение живое, просто: попросите агента «покажи список ботов в ChatPlace». Если ключ протух или введен неверно, придёт ответ про необходимость повторной авторизации. В этом случае помогает не повтор команды, а перезапуск агента — переустановите ключ и начните сессию заново.

ЛОВУШКА

Ключ API даёт полный доступ к аккаунту ChatPlace и ко всем рассылкам. Не коммитьте его в git, не вставляйте в код напрямую. Храните в переменной окружения или в файле .env и добавьте этот файл в .gitignore.

Модель данных

Чтобы формулировать задачи агенту точнее, полезно понимать структуру ChatPlace: бот содержит автоворонки (automation), автоворонка состоит из шагов (step), а каждый шаг — из блоков: сообщение, условие, действие, заметка. Когда вы просите агента «добавь шаг с проверкой подписки», он работает именно с этими сущностями.

Что умеет агент через MCP

ГРУППА	ЧТО ДЕЛАЕТ АГЕНТ
Сборка воронки	ставит готовый шаблон из каталога (подписка, контент, CRM и платежи, AI, игры, рефералы); собирает типовую воронку одним вызовом («базовая» или «проверка подписки»); либо строит вручную блок за блоком — создаёт воронку, задаёт триггеры, создаёт шаги, сообщения, инлайн-кнопки, кнопки-условия, меняет статус
Боты	регистрирует и обновляет Telegram-бота, получает карточку бота и список ботов
Живой чат	показывает список чатов и историю сообщений, берёт чат себе, закрывает его, отправляет сообщение — это передача диалога от бота человеку
Рассылки	создаёт рассылку, прогнозирует охват до отправки, показывает аналитику после
AI-агент	отдельный от воронок разговорный бот: статус, настройки, база знаний, правила по темам, публикация
Сегментация	ведёт справочник тегов и переменных
Медиа	загружает файл и возвращает id для вставки картинки в сообщение
Коммерция	каталог товаров, реферальные ссылки
Комментарии	автоответы на комментарии вне логики воронок
Аналитика	вирусность роликов в Instagram и TikTok; по воронкам — общая аналитика, сводный отчёт, аналитика по шагам

Ловушки

Это самое ценное в разделе — то, что не написано в документации ChatPlace, а видно только на практике.

- установка триггеров заменяет весь список целиком, а не добавляет к нему; если правите воронку по шагам и просите добавить ещё один триггер, легко случайно стереть уже настроенные — просите агента сначала показать текущий список триггеров, а потом присылать полный новый набор;

- воронка, созданная через MCP, появляется в статусе черновика и не срабатывает, пока вы явно не переключите статус на активный — это частая причина «бот молчит», хотя воронка вроде бы готова;
- триггеры на комментарии в Instagram привязываются к конкретным постам по их идентификаторам и несовместимы с комбинированными группами триггеров — не пытайтесь смешать триггер на пост с другими условиями в одной группе;
- для веток с проверкой подписки на Telegram-канал нужен идентификатор чата канала, а не его username — просите агента получить именно id, иначе шаг проверки не сработает;
- карточка с фото и кнопкой-ссылкой доступна только для Instagram, в Telegram такой формат не собрать;
- справочник тегов через MCP вести можно, но навесить тег на конкретного контакта нельзя — привязка тега к контакту живёт только внутри логики автоворонки; в документации ChatPlace это явно не описано, вывод сделан по опыту работы;
- ключ API выпускается только руками в кабинете, сгенерировать его через агента нельзя.

Что делать через агента, а что руками

Через агента удобно вести повторяемые операции: собирать воронки по шаблону, массово править тексты сообщений, снимать отчёты, прогонять прогноз охвата перед рассылкой. Это экономит время и не требует переключения в браузер.

Руками в вебе логичнее делать то, что происходит один раз или требует визуального контроля: первичное подключение аккаунтов Instagram и Telegram, выпуск ключа API, финальная визуальная проверка воронки перед запуском — посмотреть глазами, как выглядят кнопки и текст, прежде чем включать её на живую аудиторию.

- ✓ ключ выпущен в кабинете и подключён к агенту
- ✓ агент отвечает на запрос списка ботов
- ✓ первая воронка собрана и проверена глазами в вебе перед активацией
- ✓ ключ лежит в переменной окружения, а не в коде

Примеры промптов

Вот как это звучит в переписке с агентом:

```
Собери в ChatPlace воронку: комментарий с кодовым словом
"ГАЙД" под постом [id поста] -> сообщение в Direct со ссылкой
на материал. Черновиком, не активируй без моего подтверждения.
```

Покажи отчёт по воронке "Лид-магнит Instagram" за последние две недели: сколько вошло, сколько дошло до конца, на каком шаге отваливаются чаще всего.

Перед отправкой рассылки по тегу "тёплые" сначала спрогнозируй охват и покажи мне текст сообщения. Отправлять только после моего явного "да".

Обнови текст приветственного сообщения во всех активных Telegram-ботах на новую формулировку [текст]. Сначала покажи список ботов, которые затронешь, я подтвержу.

Связка с остальным конвейером

Так это соединяется в единый конвейер. Reel опубликован через автопубликацию, под ним появляется комментарий с кодовым словом, воронка ChatPlace подхватывает его и выдаёт материал в Direct — без вашего участия. А отчёт по этой воронке вы забираете той же командой агенту, что и метрики самих публикаций: не переключаясь между вкладками, а просто продолжая разговор в терминале.

17

17. Автопубликация

Для публикационного контура логичнее использовать PostMyPost, а не платить ChatPlace за каждый канал.

PostMyPost заявляет поддержку Instagram, Telegram, VK и MAX; для MAX канал подключается через [@PostmypostBot](#), которому выдаются права администратора. У сервиса есть семидневный тестовый период.

Принцип

Создаётся один master-пакет, но не один одинаковый пост:

- Instagram: короткий Reel, сильный первый кадр, компактная подпись, кодовое слово;
- Telegram: больше контекста, процесс, цена/условия, ссылка или контакт;

- ВК: нативное видео, описание услуги, СТА внутри ВК;
- МАХ: короткий текст, визуал и прямой следующий шаг.

Один и тот же тр4 можно использовать повторно, но подписи, обложки и СТА должны адаптироваться.

На первом этапе агент создаёт **черновики**, а не публикует сам. Автопубликацию без подтверждения включаем после 2–3 недель стабильной работы.

18

18. Контентная логика по направлениям

Основной аккаунт (пример: студия портретов на заказ)

Контентные опоры:

1. «Фото → готовое изделие».
2. Процесс: обработка, цвет, печать, натяжка, упаковка.
3. Реакция получателя и подарок с эмоцией.
4. Разбор размера, материала, цены и срока.
5. Кейсы и отзывы.
6. Ошибки исходных фотографий и как выбрать хорошую.
7. Автор в кадре: история заказа и профессиональный взгляд.

Главные СТА:

- ЗАКАЗ — расчёт заказа;
- РАЗМЕР — подбор формата;
- ПОДАРОК — идеи под событие.

Сначала достаточно одного кодового слова ЗАКАЗ, чтобы не дробить аналитику.

Второй профиль (пример: портфолио)

Контентные опоры:

1. Чистое портфолио и новые тесты.

2. Backstage съёмок и показов.
3. Движение, спорт, пластика, навыки.
4. Короткие представления и self-tape.
5. Подтверждённые достижения.
6. Доступность к проектам и контакты для букинга.

Для этого аккаунта часто важнее профессиональная полезность для того, кто принимает решение — заказчика, работодателя, агентства, — чем мемы и массовые охваты.

Если человек в кадре — несовершеннолетний, рабочие коммуникации ведите через родителя или опекуна; не публикуйте документы, точный адрес, учебное заведение, маршруты и лишние личные данные.

Архивный и коммерческий каналы (опционально)

Если у вас есть архивный/наследный аккаунт и отдельный канал коммерческой продажи оригиналов — не смешивайте их:

- наследие: биография, контекст работ, архив, выставки;
- галерея: конкретная работа, размер, техника, состояние, провенанс, цена/запрос цены и контакт коллекционера.

19

19. Качественные посты

Reel приносит охват. Текст приносит доверие. Человек посмотрел ролик, зашёл в профиль и открыл закреплённый пост — именно там он решает, специалист вы или очередной аккаунт с монтажом.

Этот раздел про то, как ставить производство текстов на тот же конвейер, что и видео: один раз описать свой голос, дальше получать от Codex/Claude Code черновики, которые нужно править, а не переписывать заново.

Агент, который не знает вашего голоса, выдаёт узнаваемый машинный текст. Тот же агент с описанным голосом и вашими фактами экономит основное время — на структуре и первом черновике.

Анатомия поста, который дочитывают

Работающий пост держится на пяти элементах. Порядок важен: каждый следующий имеет смысл только если сработал предыдущий.

ЭЛЕМЕНТ	ЧТО ДЕЛАЕТ	ПРИЗНАК, ЧТО НЕ СРАБОТАЛ
Хук	Останавливает пролистывание в первых одной-двух строках	Досмотры есть, дочитываний нет
Контекст	Объясняет, почему вы вправе это говорить	Текст читается как пересказ чужой статьи
Инсайт	Даёт вывод, до которого читатель сам не дошёл	«Согласен» в комментариях, но нет сохранений
Артефакт	Оставляет в руках вещь: промпт, схему, чек-лист, команду	Пост лайкают, но не пересылают
СТА	Переводит внимание в следующий шаг	Охват растёт, заявок нет

Из этих пяти чаще всего пропускают артефакт, и именно он сильнее прочего влияет на пересылки. Пост-мнение расходится по знакомым; пост с готовым к применению куском забирают себе.

ПРОВЕРКА

Спросите себя после написания: что читатель сможет сделать сегодня, не задав ни одного уточняющего вопроса? Если ответа нет, артефакта в посте тоже нет.

Хук: четыре детали

Первая строка решает судьбу поста. Сильный хук почти всегда содержит четыре вещи одновременно:

- 1. вход, который ломает шаблон ленты: короткая рубленая фраза, цифра или прямое признание;
- 2. обещание, которое звучит контринтуитивно для вашей темы;
- 3. личное доказательство с измеримой величиной: срок, сумма, количество, процент;
- 4. ясное указание, что читатель получит ниже.

Скелет, который удобно давать агенту как образец:

[Резкий вход]. [Контринтуитивное обещание].
[Что я сделал и с каким измеримым результатом].
Ниже — [что именно вы получите].

Хук проверяется одним вопросом: можно ли поставить эту первую строку к чужому посту по той же теме? Если можно, она слишком общая и её нужно переписать до конкретики.

СЛАБЫЙ ХУК

AI сильно меняет то, как мы работаем с текстами. Расскажу, как использую нейросети в своей работе.

СИЛЬНЫЙ ХУК

Я четыре месяца писал посты руками и был уверен, что ИИ здесь бесполезен. Оказалось, я просто не отдавал ему свой голос. Ниже — файл на страницу, после которого черновики стало можно править, а не выбрасывать.

Пять форматов, которых хватает

Не нужно изобретать новую форму под каждую тему. Рабочих форматов пять, и они закрывают почти любое сырьё.

- **Разбор.** Берёте одну задачу и показываете решение по шагам. Артефакт: последовательность действий;
- **Кейс.** Было, стало, что именно сделано, сколько заняло. Артефакт: цифры и схема решения;
- **Инструмент.** Одна вещь, которой пользуетесь сами, и честные границы её применимости. Артефакт: настройка или команда;
- **Ошибка.** Что сломалось, во что обошлось, как теперь делаете. Артефакт: правило, которое читатель заберёт;
- **Наблюдение.** Закономерность, которую заметили на нескольких проектах. Артефакт: критерий проверки.

Формат «Ошибка» недооценён. Он даёт доверие быстрее остальных, потому что человек, который публично разбирает свой промах, выглядит практиком, а не витриной.

Файл голоса: главный артефакт этого раздела

Один раз опишите свой голос, и все дальнейшие черновики станут пригодными для правки. Файл кладётся в проект рядом с бренд-фактами и подключается так же, как остальные источники правды.

brands/<brand>/voice.md

Содержимое файла — шесть блоков:

1. **Кто говорит.** Одна фраза о позиции: что вы делаете и с какой стороны смотрите на тему;
2. **Кому.** Кто читатель, какие у него задачи и какой у него уровень: чего можно не объяснять;
3. **Темы.** Три-четыре направления, вокруг которых строится весь контент;
4. **Как звучит.** Длина предложений, обращение, степень прямоты, допустимость резкости;
5. **Запрещённое.** Слова, обороты и приёмы, которых быть не должно. Самый ценный блок;
6. **Образцы.** Три-пять ваших текстов целиком.

Блок запретов работает лучше всех остальных, потому что модель охотно скатывается к среднему тону из обучающих данных. Перечислите прямо: канцелярит, восклицания, обращения вроде «друзья», эмодзи, слова «революция», «прорыв», «магия», «в эпоху ИИ», конструкции вида «это не просто X, это Y», три однородных определения подряд, выдуманные цифры.

ЛОВУШКА

Не описывайте голос по памяти. Отдайте агенту пять своих реальных текстов и попросите вывести правила из них. Собственное представление о своём стиле почти всегда расходится с тем, как вы пишете на самом деле.

Промпт для сборки файла:

```
Прочитай пять моих текстов в папке samples/.  
Выведи из них правила моего голоса: типичная длина предложения,  
как я начинаю, как заканчиваю, какие слова повторяю,  
чего в моих текстах не встречается ни разу.  
Не придумывай правила, которых нет в образцах.  
Собери brands/<brand>/voice.md по шести блокам:  
кто говорит, кому, темы, как звучит, запрещённое, образцы.  
Спорные места вынеси отдельным списком вопросов ко мне.
```

Рабочий цикл: от сырья до готового поста

Цикл повторяет логику монтажного конвейера: сначала материал, потом варианты, потом одна выбранная версия, потом проверка.

1. сложить сырьё: голосовую заметку, переписку, черновик из Apple Notes или результат проекта;

2. получить три разных угла подачи, а не три версии одного;
3. выбрать угол и получить полный текст;
4. прогнать редактуру по списку запретов;
5. получить адаптации под площадки.

Промпт первого шага:

```
Используй brands/<brand>/voice.md.  
Вот сырьё: [вставьте расшифровку или ссылку на файл].  
Предложи три угла подачи, материально разных:  
разбор, кейс и ошибка. Для каждого дай только  
хук в две строки и обещание читателю.  
Ничего не выдумывай: если фактов не хватает, скажи каких.
```

Промпт второго шага:

```
Беру второй угол. Напиши пост целиком по структуре:  
хук, контекст, инсайт, артефакт, СТА.  
Артефакт должен быть применимым сегодня и без вопросов ко мне.  
Цифры бери только из сырья. Если цифры нет, напиши «нет данных»  
вместо правдоподобной оценки.  
Соблюдай блок запрещённого из voice.md.
```

Требование пометать отсутствующие данные важнее, чем кажется. Правдоподобная выдуманная цифра в посте — самая дорогая ошибка этого конвейера: её замечают именно те читатели, ради которых всё делается.

Редактура: что вычищать всегда

Машинный текст узнаётся по устойчивому набору признаков. Держите этот список в voice.md и прогоняйте им каждый черновик.

- ✓ вводные обороты, которые ничего не добавляют: «стоит отметить», «важно понимать», «в современном мире»;
- ✓ три однородных члена подряд там, где хватает одного;
- ✓ конструкция «это не просто одно, это другое»;
- ✓ слова-усилители без содержания: «мощный», «невероятный», «революционный»;
- ✓ обобщения без субъекта: «многие считают», «эксперты говорят»;
- ✓ симметричные пары предложений одинаковой длины подряд;

- ✓ финал, который пересказывает сказанное вместо следующего шага;
- ✓ тире там, где хватает точки.

Промпт редактуры:

```
Прогони черновик по блоку запрещённого из voice.md.  
Верни таблицу: что нашёл, где, чем заменить.  
Не переписывай текст целиком, показывай точечные правки.  
Отдельно отметь все утверждения с цифрами и укажи,  
подтверждается ли каждое исходным материалом.
```

Разбор правки на одном абзаце:

до

В современном мире автоматизация играет ключевую роль. Мощные AI-инструменты открывают невероятные возможности для бизнеса, позволяя экономить время, сокращать издержки и повышать эффективность.

после

Я подключил CRM к агенту и перестал руками собирать отчёт по сделкам. Раньше это занимало сорок минут каждый понедельник. Теперь занимает одну команду.

Изменилось не оформление. В первом варианте нет ни одного факта, который можно проверить; во втором их три, и каждый принадлежит автору.

Один пост, четыре площадки

Готовый текст не копируется дословно. У площадок разная логика первого экрана и разное поведение ссылок.

ПЛОЩАДКА	ОБЪЁМ	ЧТО МЕНЯЕТСЯ
Telegram	Полный текст	Ссылка работает внутри поста, артефакт можно отдать файлом
Instagram	Сокращённый	Виден один-два первых предложения; ссылки не кликабельны, СТА ведёт в Direct по кодовому слову
LinkedIn	Средний	Решают первые три строки до сворачивания; ссылку выносят в комментарий
Threads	Ветка	Один пост — одна мысль, артефакт отдельным сообщением, призыв в конце ветки

Промпт адаптации:

```
Адаптируй утверждённый пост под Telegram, Instagram,
LinkedIn и Threads. Для каждой площадки учти:
объём, что видно до сворачивания, работает ли ссылка,
куда ведёт призыв. Смысл и факты не меняй.
Для Instagram отдельно дай кодовое слово для комментария.
```

Кодовое слово из последнего пункта — это стык с воронкой в Direct, описанной в разделе про ChatPlace. Комментарий с ним запускает выдачу артефакта и превращает читателя в заявку.

Чек-лист перед публикацией

- ✓ пост относится к одному из ваших направлений, а не к теме вообще;
- ✓ в тексте есть конкретика: цифры, названия, реальная задача;
- ✓ есть артефакт, применимый сегодня;
- ✓ написано от первого лица, включая то, что не получилось;
- ✓ ни одной цифры, которой нет в исходном материале;
- ✓ нет хайпа, эмодзи и обращений к аудитории скопом;
- ✓ первая строка не подходит к чужому посту по той же теме;
- ✓ призыв ведёт в одно конкретное место.

Восемь пунктов проходятся за две минуты и отсекают почти все слабые публикации. Поручите эту проверку агенту отдельной командой: он должен вернуть список нарушенных пунктов, а решение о публикации остаётся за вами.

20

20. Качественные карусели

Карусель — самый недооценённый формат в этом конвейере. Она сохраняется чаще Reel, живёт в ленте дольше поста и почти всегда выигрывает там, где нужно объяснить последовательность.

Главная мысль раздела: копировать чужую визуальную систему бессмысленно. Выигрывает не набор красивых слайдов, а узнаваемость — когда человек опознаёт вашу карусель по первому экрану, ещё не увидев имени. Ниже описан способ вывести такую систему из своего бренда и поручить сборку Codex/Claude Code.

ЛОВУШКА

Не пытайтесь генерировать слайды нейросетью-картинкой. Генеративные модели до сих пор не дают ни корректный логотип, ни русский текст без ошибок в буквах. Слайды верстаются кодом или в редакторе; изображение внутри слайда генерировать можно.

Шесть законов, которые держат систему

Эти правила выведены из разбора нескольких десятков сильных каруселей разных авторов. Они повторяются независимо от темы и от палитры.

- 1. Один акцентный цвет на всю систему.** Второй акцент разрушает узнаваемость быстрее любой другой ошибки;
- 2. Один сквозной элемент на каждом слайде.** Плашка, линия, номер, угловой маркер — что-то, что повторяется от слайда к слайду и склеивает их в набор;
- 3. Три шрифтовых регистра с жёсткими ролями.** Плакатный заголовок, нейтральное тело и третий голос для ремарок. Четвёртый регистр превращает вёрстку в кашу;
- 4. Финальный слайд ломает сетку.** После однообразного ритма перелом читается как сигнал «здесь действие»;
- 5. Доказательство вместо утверждения.** Хотя бы один слайд несёт скриншот, цифру или результат;
- 6. Нумерация в заголовке.** Точки-индикаторы Instagram человек не считает; «3 из 8» в самом слайде удерживает до конца.

Раскладка: у каждого слайда одна роль

Рабочий диапазон — от семи до десяти слайдов. Меньше семи не успевает раскрыть мысль, больше десяти теряет дочитывание.

слайд	роль	правило
1	Обложка-хук	Заголовок в две-четыре строки, одно слово акцентом, без объяснений и без подзаголовка
2	Боль или контекст	Узнаваемая ситуация читателя, без решения
3	Перелом	Неочевидный вывод, ради которого стоит листать дальше
4–7	Содержание	Один слайд — одна мысль. Две мысли на слайде читаются как ноль
8	Доказательство	Скриншот, цифры, результат
9	Свод	Вся схема или чек-лист целиком: этот слайд сохраняют
10	Действие	Ломает сетку, содержит кодовое слово для комментария

Слайд свода стоит собирать в первую очередь. Если он не получается, значит мысль ещё не продумана, и остальные слайды будут растекаться.

Пропорции: числа, которые переносятся на любой бренд

Размеры задаются в процентах от холста, поэтому работают при любом формате и любой палитре.

- поля: 6–8% ширины со всех сторон;
- заголовок обложки: 8–11% высоты на строку, межстрочное расстояние 0,85–0,95 от кегля;
- основной текст: 2,2–3,2% высоты. Ниже 2% в ленте не читается с телефона;
- акцентом выделяются одно-два слова, никогда не строка целиком;
- скругления: ноль по умолчанию. Скругление оправдано только когда слайд имитирует чужой интерфейс;
- тень: одна мягкая и направленная, под скриншотом или макетом. Больше теней не нужно.

Формат холста — вертикаль. Безопасный выбор 1080×1350; часть авторов собирает в 1080×1440 ради большей высоты. Проверьте кадрирование в своём аккаунте перед тем, как закладывать формат в шаблон.

БЕЗОПАСНАЯ ЗОНА

Нижние примерно 12% первого слайда перекрываются интерфейсом в части раскладок ленты. Не размещайте там ничего важного: ни последнюю строку заголовка, ни логотип.

Как вывести свою систему

Система собирается из четырёх решений, и все четыре берутся из бренда, а не из чужой карусели.

- 1. Подложка.** Бумага, чистый белый, миллиметровка, тёмный экран. Подложка задаёт настроение сильнее цвета;
- 2. Акцент.** Один цвет из фирменной палитры. Проверьте его на подложке при печати и на телефоне при низкой яркости;
- 3. Третий голос.** Чем помечаются авторские ремарки: рукописный шрифт, моноширинный, капслок с разрядкой;
- 4. Грамматика цвета.** Закрепите за каждым тоном смысл и не нарушайте: например тёмная плашка — проблема, акцентная — решение, серая — слабый вариант.

Последний пункт превращает набор слайдов в язык. Читатель быстро выучивает, что означает акцент у вас, и дальше считывает структуру слайда до того, как прочтёт текст.

Две системы на одном бренд-ядре для примера. Первая: подложка цвета бумаги с лёгким зерном, плакатный гротеск в капслоке, акцентная плашка для решения, тёмная — для проблемы. Вторая: светло-серая миллиметровка, белые карточки с тонкой обводкой, акцент только в заливках и узлах схемы, рукописные ремарки на полях. Ядро общее, но узнаются они по-разному.

Сборка: HTML в PNG

Слайды собираются тем же способом, которым сверстан этот гайд: один HTML-файл и headless-браузер. Отдельный инструмент не нужен, Chrome уже стоит на машине.

Каждый слайд — секция фиксированного размера:

```
<section class="slide" data-n="01">
  <h1>Заголовок с <em>акцентом</em></h1>
</section>
```

Размер задаётся в CSS один раз, дальше все слайды наследуют сетку, поля и типографику.

Готовый рендер входит в стартовый пакет. Шесть стилей, каждый со своей палитрой, типографикой и набором блоков:

```
npm run carousel:logos # один раз, нужен интернет
npm run carousel      # посмотреть все шесть стилей
npm run carousel -- b  # только один стиль
```

Это демонстрация стилей на готовых текстах. Работа над своей каруселью идёт иначе — через заготовку поста, об этом ниже.

Логотипы сервисов в архив не входят: это чужие товарные знаки, поэтому они скачиваются отдельной командой при первом запуске. Шрифты, наоборот, лежат в архиве — все четыре распространяются по открытой лицензии, а рендер обязан давать одинаковый результат на любой машине.

Для мелкого текста бывает полезен двойной масштаб с последующим уменьшением:

```
node carousel/render.mjs a --scale=2
```

Это тот же приём, что применяется при подготовке растровых макетов под печать.

Что даёт такой способ по сравнению с ручной вёрсткой в редакторе: правка текста на всех слайдах разом, повтор набора под новую тему за минуту, отсутствие расползающихся отступов и возможность поручить пересборку агенту.

ЛОВУШКА

Не задавайте координаты текста вручную. Как только заголовок станет на строку длиннее, он наедет на соседний блок. Раскладка считается потоком: заголовок, отступ, содержание, подпись. Проверяйте шаблон самым длинным заголовком, который допускаете.

Что здесь делает агент

Разделение простое: агент отвечает за текст и раскладку по ролям, шаблон отвечает за внешний вид. Смешивать эти зоны не нужно.

Промпт на структуру:

Используй `brands/<brand>/voice.md` и правила каруселей.
Тема: [тема]. Материал: [ссылка на файл или текст].
Собери структуру на 9 слайдов по ролям:
обложка, боль, перелом, четыре слайда содержания,
доказательство, свод, действие.
Для каждого слайда верни: роль, заголовок,
текст до 20 слов, какое слово выделить акцентом.
Слайд свода собери первым и покажи отдельно:
если мысль в него не помещается, скажи об этом
вместо того, чтобы дробить её на слайды.

Промпт на сборку:

Заведи пост из шаблона стиля В, слаг «конвейер»,
название «Конвейер каруселей». Подставь утверждённые тексты
в `slides.mjs`, не трогая разметку и CSS.
Отрендери и собери контактный лист.
Проверь по нему: длина заголовков помещается в три строки,
акцент выделен на каждом слайде максимум в двух словах,
в нижние 12% первого слайда ничего не попало.
Если что-то переполнено — почини и пересобери,
покажи мне только чистый лист.

Последняя строка важнее остальных. Контактный лист — это не отчёт в конце, а инструмент цикла: агент рендерит, смотрит, чинит и рендерит снова, пока переполнения не исчезнут. Показывать вам первую попытку смысла нет.

Цикл: шаблон, пост, лист

Стили живут как шаблоны и под конкретную тему не правятся. Каждый пост получает собственную копию вёрстки, поэтому эксперименты с одной каруселью не ломают остальные и не тянутся в следующую.

```
npm run carousel:new -- b конвейер "Конвейер каруселей"  
# правите тексты в carousel/posts/конвейер/slides.mjs  
npm run carousel -- carousel/posts/конвейер/slides.mjs  
npm run carousel:sheet -- конвейер-
```

Готовые слайды складываются в `carousel/carousels/Конвейер каруселей/` с читаемыми именами вида `03 этап 1.png`. Опубликовать можно прямо оттуда: переименовывать и сортировать вручную не нужно.

Последняя команда собирает все слайды серии в одну картинку. Дальше цикл повторяется — правка, рендер, лист — пока лист не станет чистым.

ПРАВИЛО

Карусель не показывают никому, пока контактный лист не просмотрен глазами. По одному слайды почти всегда выглядят приемлемо: провал виден только на серии целиком. Ритм, чередование плотных и свободных слайдов, работу акцента и случайно одинаковые слайды подряд нельзя оценить иначе.

Фотографии для стилей, где они нужны, берутся из отдельной папки по индексу и тону, а не по имени файла. Слайд просит «первый тёмный кадр», и набор можно менять, не трогая вёрстку. Внутри тона кадры отсортированы от самых тёмных: чем темнее кадр, тем меньше на нём мешающих деталей, поэтому первые индексы достаются обложке и слайдам с длинным текстом.

Удобный способ вести этот набор — завести в приложении Фото отдельный альбом и держать его источником правды: понравился кадр, положили в альбом, выгрузили в проект командой из раздела про источники материалов. Тогда пополнение библиотеки не требует ни переименования файлов, ни правок в слайдах.

Чего в сильных каруселях не бывает

Список признаков, по которым видно наспех собранный набор. Держите его в правилах проекта и прогоняйте по контактному листу.

- ✓ градиенты и глянцевые тени;
- ✓ стоковые трёхмерные иконки;
- ✓ эмодзи в заголовках;
- ✓ второй акцентный цвет, появившийся к середине набора;
- ✓ все элементы отцентрованы на каждом слайде;
- ✓ карточки одного размера и радиуса подряд от слайда к слайду;
- ✓ схемы, повторяющие вид стандартных редакторов презентаций;
- ✓ текст, набранный кеглем мельче 2% высоты холста;
- ✓ последний слайд, визуально не отличающийся от предыдущих.

Каждый пункт этого списка встречается ровно там, где слайды собирали по одному, без взгляда на набор целиком. Отсюда и требование смотреть контактный лист перед публикацией.

21. Ваш семидневный план внедрения

Ниже — план на семь рабочих дней, который можно пройти самостоятельно, в своём темпе: один день на один блок, или быстрее, если у вас уже есть техническая база. Если вы предпочитаете готовое сопровождение вместо самостоятельного прохождения, LVMN предлагает платное внедрение под ключ по этой же структуре — написать можно в Telegram @lyaminvl <<https://t.me/lyaminvl>>.

До старта

Перед тем как начать, соберите:

- информацию об операционной системе вашего компьютера;
- 20–30 исходных фото/видео по одному реальному проекту;
- один музыкальный трек, на который есть право использования;
- актуальный прайс, сроки, доставку и FAQ;
- логотип, цвета, шрифты или 5 визуальных референсов;
- 5–10 Reels, стиль которых вам нравится;
- скриншоты Instagram Insights за 90 дней;
- порядок обработки заказа вручную;
- доступы подключайте сами, на своём устройстве; пароли не храните и не пересылайте в чатах/мессенджерах.

День I — установка и рабочая папка

- проверить аккаунт и доступные лимиты: ChatGPT-тариф для Codex либо подписку Claude для Claude Code;
- установить/проверить Codex или Claude Code;
- проверить Node, FFmpeg и Python;
- создать `content-os` ;
- настроить `.env` и `.gitignore` ;
- создать файл инструкций проекта: `AGENTS.md` для Codex или `CLAUDE.md` для Claude Code;
- показать состояние окружения: `codex doctor` и `/status` , `/usage` , `/skills` в Codex; `claude doctor` и те же `/status` , `/usage` в Claude Code;
- результат дня: агент видит тестовую папку и создаёт manifest.

День 2 — бренд и разбор исходников

- заполнить studio-brand ;
- зафиксировать цены, сроки, запреты и СТА;
- запустить транскрипцию, кадры и contact sheet;
- результат дня: полный индекс исходников и три концепции первого Reel.

День 3 — первый динамический Reel

- утвердить концепцию;
- создать edit_plan.json ;
- добавить beat-sync, hook, «до/после», процесс и СТА;
- отрендерить 2–3 версии;
- результат дня: первый готовый Reel и зафиксированный визуальный preset.

День 4 — talking-head

- создать сценарий и суфлёр;
- записать короткое видео;
- автоматически убрать паузы, добавить субтитры и B-roll;
- результат дня: второй рабочий формат.

День 5 — контент-план и переиспользование

- создать рубрики на четыре недели;
- собрать недельный пакет из существующих материалов;
- адаптировать один master-пост под Instagram/TG/MAX/BK;
- подключить PostMyPost в тестовом режиме;
- результат дня: публикационные черновики и понятный календарь.

День 6 — заявочная воронка

- подключить основной аккаунт к ChatPlace;
- создать воронку ЗАКАЗ ;
- настроить теги, поля и takeover;

- проверить со второго аккаунта;
- результат дня: тестовая квалифицированная заявка приходит вам.

День 7 — закрепление и передача

- повторить весь цикл самостоятельно;
 - разобрать ошибки;
 - подключить second-brand как второй профиль;
 - собрать SOP и список команд;
 - определить следующий спринт: аватар, новые стили, аналитика, автопубликация;
 - результат дня: вы самостоятельно запускаете новый Reel из новой папки.
-

22

22. Как будет устроено сопровождение

Вы можете пройти этот семидневный план полностью самостоятельно, день за днём, или встроить его в работу с кем-то, кто вам помогает технически. Логика одна и та же в обоих случаях.

Каждый рабочий день должен заканчиваться одним измеримым результатом и одной небольшой практической задачей на ваших материалах.

Формат каждого дня:

1. короткая формулировка «что делаем сегодня»;
2. инструкция или короткая запись экрана;
3. одна практическая задача на ваших материалах;
4. проверка результата;
5. текстовое резюме «что теперь работает»;
6. обновление SOP.

Оптимально оставить два контрольных окна:

- подтверждение концепции;
- подтверждение финального ролика/публикации.

Остальные технические шаги агент выполняет сам.

23

23. Критерии готовности

Система считается рабочей, если вы можете самостоятельно:

- положить новую пачку материалов в inbox ;
- вызвать \$reel ;
- получить три концепции;
- выбрать одну;
- получить корректный mp4 и обложку;
- попросить правку обычным языком;
- получить платформенные версии текста;
- создать публикационные черновики;
- объяснить, откуда приходят заявки ChatPlace;
- найти инструкцию и повторить процесс через неделю.

Технические критерии первого Reel:

- 1080×1920, H.264/AAC, 30 fps;
- нет чёрных кадров и обрезанного текста;
- субтитры читаемы и не перекрываются интерфейсом Instagram;
- факты, цена и срок подтверждены;
- первые 1–2 секунды содержат понятный хук;
- есть один СТА;
- музыка легальна для выбранного контура публикации;
- опубликованная версия сохранена вместе с исходным edit_plan.json .

24. Метрики

Контент

- удержание первых 3 секунд;
- среднее время просмотра;
- доля досмотров;
- сохранения и отправки;
- переходы в профиль;
- комментарии с кодовым словом.

Воронка

- кодовое слово → первое сообщение;
- первое сообщение → нажатие кнопки;
- нажатие → заполненные данные;
- заполненные данные → ручной диалог;
- ручной диалог → расчёт;
- расчёт → заказ/предоплата.

Второй профиль

- просмотры профиля;
- сохранения портфолио;
- входящие от агентств/партнёров/брендов;
- запросы self-tape;
- реальные букинги.

После 10–15 публикаций skill analytics должен выявлять не «лучшее время вообще», а паттерны конкретных тем, хуков, длительности и СТА.

25. Безопасность и приватность

- Все ключи хранятся в `.env`, который исключён из git.
 - В `SKILL.md` не должно быть паролей, токенов, карт и персональных идентификаторов.
 - Оригиналы исходников не перезаписываются.
 - Автопубликация и отправка сообщений включаются только после тестов.
 - FFmpeg, Remotion и локальный Whisper обрабатывают медиа на компьютере. Но промпты и выбранные для контекста файлы агента обрабатываются по правилам вашего ChatGPT-аккаунта или API-организации.
 - При использовании облачного Whisper/Groq извлечённые данные отправляются внешнему провайдеру; чувствительные материалы лучше транскрибировать локально.
 - Для цифрового двойника используется только собственное лицо/голос с явным согласием.
 - Не загружать паспорта, договоры с персональными данными, адреса и детские документы в брендовый контекст.
 - Подписка ChatGPT и OpenAI API — разные способы оплаты. Для обычной работы выбираем вход через ChatGPT; API-ключ подключаем только осознанно для отдельной автоматизации.
-

26. Ответы на ваши вопросы

«Может ли Codex/Claude Code сам смонтировать стильный Reel?»

Да, если агент подключён к FFmpeg/Remotion и получает транскрипт, кадры и данные музыки. Сам агент принимает решения и запускает монтаж, а финальный mp4 собирают локальные инструменты.

«Чем Codex и Claude Code отличаются от обычного чата с моделью?»

Обычный чат — ChatGPT в браузере или Claude в приложении — удобен для обсуждения и отдельных текстов. Он не видит ваши файлы и не может ничего запустить.

Codex и Claude Code работают внутри выбранной папки: читают файл инструкций (`AGENTS.md` у Codex, `CLAUDE.md` у Claude Code), вызывают skills, запускают FFmpeg/Remotion, создают файлы и проверяют фактический результат. Поэтому весь production-контур фиксируем в агенте, а не в чате.

«Может ли он сам выбрать трек?»

Может предложить настроение и выбрать из локальной лицензированной библиотеки. Трендовые Instagram-треки лучше выбирать внутри Instagram и добавлять вручную из-за лицензий и региональных ограничений.

«Может ли он сделать хуки, мемные вставки и субтитры?»

Да. Хуки и подписи создаёт агент; субтитры дают Whisper + Remotion; мемы и B-roll берутся из собственной разрешённой библиотеки или генерируются отдельным инструментом.

«Можно ли сделать аватара?»

Да. Агент пишет сценарий и управляет HeyGen/Captions, затем монтирует получившийся аватарный клип вместе с реальными материалами и подписями.

«Можно ли автоматически публиковать везде?»

Да, через PostMyPost и/или официальные API, но сначала создаются черновики. Один и тот же текст без адаптации во все соцсети не отправляем.

«Нужен ли ChatPlace?»

Да, для Instagram-воронки и заявок. Для монтажа и кросспостинга он не нужен. Начните с одного основного аккаунта.

27. Что нужно согласовать до старта

Прежде чем приступать к Дню 1, ответьте себе на эти вопросы — это сэкономит время на первых днях внедрения.

1. Какая у вас операционная система?
2. Есть ли права администратора на компьютере для установки FFmpeg/Node/Python?
3. Если у вас будет второй профиль с другим человеком в кадре — он совершеннолетний?
4. Какой аккаунт сейчас подключён к ChatPlace (если уже подключали) и какой тариф активен?
5. Есть ли ВК-сообщество, ВК-канал или оба ресурса, и кто владелец?
6. Какие цены, сроки, города доставки и способы оплаты можно публиковать?
7. Есть ли согласия клиентов на показ исходных фотографий и готовых работ?
8. Какие треки можно использовать коммерчески?
9. Какие три Reels за последний квартал дали лучшие заявки, а не просто просмотры? Если у вас пока нет опубликованных Reels — пропустите этот пункт и вернитесь к нему после первых публикаций.
10. Где сейчас хранится медиатека и сколько в ней данных?

28. Актуальные официальные источники

Ссылки разделены по инструментам. Читать нужно тот блок, который соответствует вашему выбору из раздела 2; остальные приведены для сравнения.

Codex

- Быстрый старт с ChatGPT Desktop и режимом Codex <<https://learn.chatgpt.com/docs/quickstart>>
- Тарифы, лимиты и доступность Codex <<https://learn.chatgpt.com/docs/pricing>>
- Вход через ChatGPT и отдельный вход с API-ключом <<https://learn.chatgpt.com/docs/auth>>
- Установка и использование Codex CLI <<https://learn.chatgpt.com/docs/codex/cli>>
- Постоянные инструкции проекта в AGENTS.md <<https://learn.chatgpt.com/docs/agent-configuration/agents-md>>

- Skills в Codex и структура `.agents/skills/<name>/SKILL.md` <<https://learn.chatgpt.com/docs/build-skills>>
- Импорт локальной настройки из Claude Code в Codex <<https://learn.chatgpt.com/docs/import>>
- Разрешения и песочница Codex <<https://learn.chatgpt.com/docs/sandboxing>>

Claude Code

- Обзор Claude Code и что он умеет <<https://docs.claude.com/en/docs/claude-code/overview>>
- Установка и первый вход <<https://docs.claude.com/en/docs/claude-code/setup>>
- Быстрый старт: первая задача в папке проекта <<https://docs.claude.com/en/docs/claude-code/quickstart>>
- Постоянные инструкции проекта в `CLAUDE.md` <<https://docs.claude.com/en/docs/claude-code/memory>>
- Agent Skills и структура `.claude/skills/<name>/SKILL.md` <<https://docs.claude.com/en/docs/claude-code/skills>>
- Подключение MCP-серверов, в том числе ChatPlace <<https://docs.claude.com/en/docs/claude-code/mcp>>
- Настройки и разрешения на команды <<https://docs.claude.com/en/docs/claude-code/settings>>
- Тарифы и расход лимитов <<https://docs.claude.com/en/docs/claude-code/costs>>

Монтаж, субтитры и аватары

- CapCut Auto Cut и beat-sync <<https://www.capcut.com/help/auto-cut-in-capcut>>
- Captions AI Edit <<https://captions.ai/features/edit-with-ai>>
- Captions AI Twin <<https://captions.ai/features/create-ai-twin>>
- HeyGen Digital Twin <<https://help.heygen.com/en/articles/12089286-create-your-first-digital-twin-video-avatar-with-avatar-iv>>

Публикация и воронка

- ChatPlace: актуальные тарифы на 24 июля 2026 <<https://help.chatplace.io/en/articles/6763310>>
- ChatPlace: запуск автоматизации из комментария <<https://help.chatplace.io/en/articles/12127531>>
- PostMyPost: подключение канала MAX <<https://postmypost.io/ru/resources/avtoposting-v-kanaly-max-podklyuchayte-bez-yurlitsa-i-vedite-kanal-sistemno>>
- MAX Bot API <<https://dev.max.ru/docs-api>>

29. Итоговая модель работы

Вся система сводится к одному понятному принципу:

Вам не нужно изучать пятнадцать сервисов. Соберите один рабочий центр в Codex или Claude Code. Вы будете складывать материалы в папку и ставить задачу обычным языком. Агент сам разберёт исходники, предложит идеи, соберёт монтаж, подготовит тексты и публикации. Внешние сервисы остаются только там, где без них нельзя: ChatPlace — для заявок из Instagram, PostMyPost — для публикаций, HeyGen/Captions — для цифрового двойника. За первую неделю вы соберёте рабочий MVP на своих материалах и научитесь повторять процесс самостоятельно.

30. С чего начинается работа

Начинаем с единого рабочего процесса на Codex или Claude Code:

Не распыляйтесь между разными нейросетями. Вместо этого постройте единый рабочий процесс на выбранном агенте.

Вы будете складывать фото, видео и музыку в одну папку и обычными словами писать, какой результат нужен. Агент будет сам разбирать материалы, придумывать концепции и хуки, выбирать сильные фрагменты, создавать монтажный план, собирать ролик, добавлять субтитры и готовить версии публикаций для разных соцсетей.

Отдельные сервисы подключаются только там, где они действительно нужны: ChatPlace — для заявок из Instagram, PostMyPost — для публикаций, HeyGen или Captions — для цифрового двойника. Основная работа и управление остаются в агенте.

В первую неделю начните с основного аккаунта. Первая измеримая цель — собрать на своих материалах рабочий Reel, который вы сможете самостоятельно повторить тем же процессом. После этого сделайте разговорный формат, подключите публикационные черновики и воронку по кодовому слову «ЗАКАЗ».

Прежде чем начинать, соберите:

1. Какая у вас операционная система — Windows или macOS.

2. Одну папку с материалами по конкретному проекту: исходное фото, процесс и готовая работа.
3. Актуальные цены, сроки, доставку и ответы на частые вопросы.
4. Пять Reels, визуальный стиль которых вам нравится.
5. Скриншоты статистики Instagram за последние 90 дней.

Пароли и платёжные данные никому пересылать не нужно. Все сервисы вы подключаете самостоятельно на своём устройстве, следуя точным шагам из этого руководства.

31. Технический стартовый пакет

Прежде чем переходить к Дню 1 семидневного плана, полезно собрать базовую версию проекта. Тогда сама работа с семью днями начнётся сразу с ваших материалов, а не с ожидания, пока техническое ядро создаётся с нуля. Это можно сделать самому, если у вас есть базовые технические навыки, либо заказать как отдельную платную услугу настройки у LVMN — Telegram @lyaminvl <<https://t.me/lyaminvl>>.

Часть этого списка уже собрана за вас: `Content05-starter` в комплекте с гайдом даёт рабочие команды `ingest`, `analyze`, `render` и `qa`, реальное определение битов музыки и автоматическую нарезку фото под них (`--auto-cut-to-beats`, `preset studio-clean`), schema для `edit_plan.json` и два примера job-папок — распакуйте архив, выполните `npm install` и переходите к пункту про брендовые профили ниже. Транскрипция речи и `preset talking-head-clean` — это то, что осталось доделать под ваш стиль; в архиве есть заготовки и README с описанием, что именно нужно дописать.

Кроме конвейера Reels в архив входят ещё три модуля, закрывающие остальные разделы этого гайда:

ПАПКА	ЧТО ДАЁТ	КОМАНДЫ
<u>carousel/</u>	Шесть стилей каруселей, рендер в PNG 1080×1350 через Chrome, загрузчик логотипов, шрифты	<u>npm run carousel:logos</u> , <u>npm run carousel</u>
<u>posts/</u>	Шаблон файла голоса, цепочка промптов, чек-лист публикации и механическая проверка текста	<u>npm run post:lint</u>
<u>sources/</u>	Выгрузка заметок и медиа из Apple Notes и Apple Photos в папку задания	<u>npm run notes:export</u> , <u>npm run photos:export</u>

Ни один из модулей не добавляет npm-зависимостей: пакет по-прежнему ставится без единой внешней библиотеки и работает на том, что уже есть на машине.

Минимальный технический пакет

- чистый fork стартового [reels](#) -pipeline без демо-названий и цветов;
- команда [npm run ingest <job-dir>](#);
- команда [npm run analyze <job-dir>](#);
- команда [npm run render <edit-plan>](#);
- команда [npm run qa <output-file>](#);
- schema для [request.md](#);
- schema для [edit_plan.json](#);
- один preset [studio-clean](#);
- один preset [talking-head-clean](#);
- skill [\\$reel](#);
- базовый [AGENTS.md](#);
- [.env.example](#) без секретов;
- [SETUP.md](#) для macOS и Windows;
- тестовый набор обезличенных media-файлов;
- проверка, что всё работает с путями, содержащими кириллицу и пробелы.

Что настраивается по мере прохождения 7 дней

- заполнение вашего бренд-контекста;
- настройка цветов и шрифтов;

- первый реальный `edit_plan.json`;
 - настройка ChatPlace;
 - подключение аккаунтов PostMyPost;
 - первый реальный talking-head;
 - освоение ежедневного использования.
-

32

32. Установка агента: Codex или Claude Code

Основной интерфейс для вас — приложение ChatGPT Desktop с выбранным режимом **Codex**. CLI устанавливаем рядом: он нужен для диагностики, импорта и воспроизводимых команд.

ChatGPT Desktop

1. Установить официальное приложение ChatGPT для macOS или Windows.
2. Войти под своим ChatGPT-аккаунтом.
3. В переключателе выбрать **Codex**.
4. Открыть папку `content-os`.

Доступ к Codex есть на тарифах ChatGPT, включая Free; отличаются доступные модели и лимиты. Для регулярного рендера проверьте реальный расход на своём тарифе до покупки дополнительных кредитов.

macOS и Linux: Codex CLI

```
curl -fsSL https://chatgpt.com/codex/install.sh | sh
codex --version
codex login
codex login status
codex doctor
```

Windows PowerShell: Codex CLI

```
irm https://chatgpt.com/codex/install.ps1 | iex
codex --version
codex login
codex login status
codex doctor
```

Альтернатива через npm

```
npm install --global @openai/codex
codex --version
```

Этот вариант используем, если standalone installer недоступен, а Node.js уже установлен.

Первый вход и проверка

1. Выполнить `codex login`.
2. Выбрать `Sign in with ChatGPT`.
3. Завершить вход в открывшемся браузере.
4. Выполнить `codex login status`.
5. Открыть папку `content-os` и запустить `codex`.
6. Внутри Codex CLI выполнить `/status`, `/usage` и `/skills`.
7. Выполнить `/init` только если `AGENTS.md` ещё не создан.

Не выбирайте API key для обычной работы по подписке. Вход с API-ключом включает отдельную оплату OpenAI API и нужен в основном для CI или другой программной автоматизации.

Если на компьютере остались локальные настройки Claude

Если вы раньше работали с Claude Code и хотите перенести настройки, это не мешает начать работу с Codex заново. В ChatGPT Desktop откройте **Settings → Import** и выберите Claude Code. В CLI можно выполнить `/import` и выбрать Claude Code. Codex перенесёт поддерживаемые инструкции в `AGENTS.md`, `skills`, проекты и настройки, не удаляя исходные файлы.

После импорта всё равно проверяем пути, разрешения, токены и команды: старые Claude-настройки не должны автоматически получать доступ к публикации или секретам.

Установка Claude Code

Если вы выбрали Claude Code вместо (или вместе с) Codex — установка занимает те же пару минут. Основной интерфейс — приложение **Claude Desktop** с вкладкой «Code»; CLI устанавливаем рядом для диагностики и повторяемых команд, точно как с Codex CLI выше.

MACOS, LINUX, WSL: НАТИВНАЯ УСТАНОВКА

```
curl -fsSL https://claude.ai/install.sh | bash
claude --version
claude doctor
```

WINDOWS POWERSHELL: НАТИВНАЯ УСТАНОВКА

```
irm https://claude.ai/install.ps1 | iex
claude --version
claude doctor
```

АЛЬТЕРНАТИВА ЧЕРЕЗ NPM (НУЖЕН NODE.JS 22+)

```
npm install -g @anthropic-ai/claude-code
claude --version
```

Есть и другие пакетные менеджеры, если вы уже ими пользуетесь: `brew install --cask claude-code` на macOS или `winget install Anthropic.ClaudeCode` на Windows.

ПЕРВЫЙ ВХОД И ПРОВЕРКА

1. Открыть терминал в папке content-os.
2. Выполнить `claude`.
3. Завершить вход в открывшемся окне браузера.
4. Выполнить `claude --version` и `claude doctor` для проверки установки.

Claude Code требует платного тарифа — Pro, Max, Team, Enterprise или Console (API-биллинг по факту использования). Бесплатный план claude.ai доступа к Claude Code не даёт — это отдельная опция, которую нужно включить или подключить.

Если вам не нужен терминал вообще — приложение **Claude Desktop** (macOS, Windows, Linux) даёт тот же Claude Code через вкладку «Code» прямо в интерфейсе, без единой команды в CLI. Это прямой аналог связки ChatGPT Desktop + режим Codex, описанной выше.

33

33. Установка монтажных зависимостей

Точные команды зависят от операционной системы. Проверяем не факт установки, а фактическую работу каждой команды.

Проверка

```
ffmpeg -version
ffprobe -version
node --version
npm --version
python3 --version
```

macOS через Homebrew

```
brew install ffmpeg node python
```

Windows через WinGet

```
winget install Gyan.FFMpeg
winget install OpenJS.NodeJS.LTS
winget install Python.Python.3.12
```

После установки закрыть и заново открыть Terminal/PowerShell, затем повторить проверку версий.

Node-зависимости вашего проекта

```
npm install remotion @remotion/cli @remotion/bundler react react-dom
npm install zod gray-matter sharp dotenv
npm install -D typescript tsx @types/node @types/react
```

Python-зависимости для музыки

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install librosa soundfile numpy
```

На Windows PowerShell активация окружения:

```
.venv\Scripts\Activate.ps1
python -m pip install librosa soundfile numpy
```

Версии фиксируются в package-lock.json и requirements.txt. Нельзя оставлять production-pipeline на плавающих зависимостях без проверки.

34

34. Готовый каркас AGENTS.md

Этот текст используется как основа в корне вашего проекта. Codex читает его перед началом работы; здесь остаются только постоянные правила, а конкретная задача хранится в request.md:

ContentOS

Purpose

This workspace creates social content for your brands. Codex is the central operator. It analyzes source media, proposes concepts, creates edit plans, renders media through local tools, validates outputs, and prepares platform-specific publication drafts.

Brands

- `studio`: photo-based custom product (example: portraits); primary goal is qualified orders.
- `second`: portfolio profile for a second audience; primary goal is bookings/inquiries.
- Never combine brand facts, tone, pricing, CTAs, or analytics.

Sources of truth

- Brand facts: `brands/<brand>`.
- Prices: `brands/<brand>/prices.md`.
- Approved offers: `brands/<brand>/offer.md`.
- Media ownership and consent: `brands/<brand>/rights.md`.
- Previous results: `analytics/content-results.csv`.

If a price, deadline, achievement, location, testimonial, or product fact is not present in a source-of-truth file, do not invent it. Mark it as missing.

Workflow

1. Validate the job folder and `request.md`.
2. Preserve originals. Write generated files only to `work/` and `output/`.
3. Create `manifest.json`, transcript, frames, contact sheet, and beats.
4. Propose three concepts unless the request explicitly says `mode: auto`.
5. Wait for concept approval.
6. Create or update `edit\_plan.json`.
7. Render through the project scripts.
8. Run QA and save `qa.md`.
9. Show the output path and summarize decisions.
10. Never publish without explicit approval.

Output requirements

- Vertical Reel: 1080x1920, 30 fps, H.264 video, AAC audio.
- Keep critical text inside platform safe zones.
- Captions must be proofread against the transcript.
- Every commercial Reel must have one clear CTA.
- Create platform-specific copy; do not blindly duplicate one caption everywhere.

Safety

- Never print or copy secrets from `.env`.
- Never put access tokens into `SKILL.md`, reports, or chat responses.
- Do not overwrite or delete original media.
- Do not publish media without confirmed rights and consent.
- If a person featured in `second` is a minor, do not expose school, address, routes, documents, or private contacts.
- Human approval is required for prices, public claims, avatar generation, and publishing.

Commands

```
- `npm run ingest -- <job-dir>`  
- `npm run analyze -- <job-dir>`  
- `npm run render -- <edit-plan>`  
- `npm run qa -- <output-file>`  
- `npm test`  
- `npm run typecheck`
```

После заполнения брендов английскую техническую часть можно оставить: важнее однозначность правил, чем язык файла. Публикуемые тексты и ответы агента остаются на русском.

Для Claude Code: этот же файл используется как есть — без единой правки контента. Единственное отличие — имя: сохраните его как CLAUDE.md в корне проекта (Claude Code не читает AGENTS.md автоматически, хотя часть новых agentic-инструментов на рынке действительно движется к AGENTS.md как общему стандарту — на всякий случай называйте файл именно CLAUDE.md). Если хотите вести один файл сразу для обоих инструментов, можно держать CLAUDE.md симлинком на AGENTS.md (`ln -s AGENTS.md CLAUDE.md`) — оба инструмента просто читают обычный Markdown-файл по имени, так что симлинк — рабочий вариант, а не хак.

35

35. Готовый каркас \$reel -skill

Путь для Codex: .agents/skills/reel/SKILL.md. В Codex skill можно выбрать через /skills или явно упомянуть как \$reel.

```
---  
name: reel  
description: Create a complete vertical Reel from a job folder: analyze media, propose concepts, build an edit plan, render, run QA, and prepare delivery files. Use for any request to create or revise a Reel.  
---  
  
# Reel production workflow  
  
## Input  
  
Accept either:  
  
- a path to a job folder under `inbox/`; or  
- the newest unprocessed folder in `inbox/` if the user explicitly says to use it.  
  
The job should contain `request.md` and one or more of:  
  
- `video/`  
- `photo/`
```

- `music/`

Rules

- Read `AGENTS.md` and the selected brand folder first.
- Never mix assets or facts from different brands.
- Never edit originals in `inbox/`.
- Do not invent prices, deadlines, testimonials, achievements, or rights.
- Do not publish anything.
- Use local scripts for media operations. Do not simulate successful rendering.

Phase 1: validate and ingest

1. Parse `request.md`.
2. Confirm the brand exists.
3. Check that media files are readable.
4. Run `npm run ingest -- <job-dir>`.
5. Review `manifest.json` and report blocking input problems.

Phase 2: analyze

1. Run `npm run analyze -- <job-dir>`.
2. Read transcript, frames, contact sheet, scenes, and beats.
3. Identify the strongest available proof, transformation, emotion, and CTA moment.
4. Reject unusable or duplicate shots in the analysis; do not delete them.

Phase 3: concepts

Create three materially different concepts:

1. commercial and clear;
2. emotional/story-driven;
3. dynamic/experimental.

For each include:

- hook;
- viewer promise;
- scene order;
- text overlays;
- music logic;
- CTA;
- source limitations.

Save them to `work/<job>/concepts.md` and ask the user to select one.

Phase 4: plan and render

After approval:

1. Create a schema-valid `edit\_plan.json`.
2. Run `npm run render -- <edit-plan>`.
3. If rendering fails, diagnose and fix the cause. Never claim success without an mp4.
4. Run `npm run qa -- <output-file>`.
5. Review the final contact sheet and QA report.

Phase 5: delivery

Return:

```
- final mp4 path;
- cover path;
- captions path;
- `edit_plan.json` path;
- QA result;
- short list of creative decisions;
- known limitations;
- suggested next action.
```

If QA fails, do not present the file as final.

Revisions

Translate natural-language feedback into the smallest possible edit-plan change.
Preserve the prior version and render a new numbered output.

Этот skill должен быть коротким диспетчером. Сложная логика, схемы и команды живут в коде и reference-файлах, а не в одном гигантском промпте.

Для Claude Code: тело skill выше (frontmatter с name / description , Input/Rules/Phase 1-5/Revisions) переносится дословно, без единой правки. Меняется только путь: сохраните тот же файл как .claude/skills/reel/SKILL.md . Вызов тоже чуть отличается — Codex вызывается явным упоминанием \$reel , Claude Code либо автоматически подхватывает skill по полю description , когда запрос совпадает с ним по смыслу, либо вы вызываете его явно командой /reel (для этого достаточно завести файл-команду с тем же именем — механизм описан в документации Claude Code по slash-командам).

36

36. Остальные skills: минимальные спецификации

Как и с \$reel в разделе 31: для Claude Code все skills ниже собираются по тому же принципу и с тем же телом SKILL.md, только в папке .claude/skills/<name>/ вместо .agents/skills/<name>/ . Повторять эту оговорку под каждым skill не будем.

\$content-director

Вход:

- выбранный бренд;
- цель периода;
- список доступных материалов;

- результаты прошлых публикаций;
- ограничения времени и бюджета.

Результат:

- 3–5 контентных опор;
- приоритетные идеи на неделю;
- связь каждой идеи с бизнес-целью;
- какой исходник нужен;
- формат;
- СТА;
- метрика успеха.

Не должен генерировать месяцы контента без проверки наличия материалов.

\$talking-head

Результат:

- три хука;
- утверждённый сценарий;
- версия для суфлёра;
- shot list для B-roll;
- правила записи;
- монтажный план после появления записи;
- готовый Reel и .srt .

\$repurpose

Создаёт отдельные блоки:

- instagram.md ;
- telegram.md ;
- max.md ;
- vk.md ;
- metadata.json с названием, СТА, ссылкой на master-файл и датой.

\$publish

- работает только с утверждённой папкой output/ ;
- проверяет наличие approved: true ;
- сначала создаёт черновики;
- показывает аккаунты, даты и файлы;
- публикует только после отдельного подтверждения;
- сохраняет внешние publication IDs.

\$analytics

- принимает экспорт или вручную заполненные метрики;
 - не делает вывод по одному ролику;
 - сравнивает тему, хук, длительность, визуальный preset и СТА;
 - отделяет просмотры от заявок;
 - записывает наблюдения, но не меняет брендовые факты.
-

37

37. Техническое ТЗ на развитие стартового reels -pipeline

Блок А. Новый входной формат

Если базовый pipeline требует один faceVideo и опциональный screencastVideo , нужно добавить job-folder и manifest.

Новые сущности:

```
JobRequest
MediaAsset
MediaAnalysis
BeatMap
Scene
TimelineItem
TextOverlay
```

Блок В. Ingest

Добавить:

- рекурсивное обнаружение media;
- ffprobe JSON;
- SHA-256 и поиск дублей;
- ориентацию и rotation metadata;
- duration/fps/codec/audio channels;
- безопасное CFR-прокси;
- сохранение связи original → proxy.

Блок С. Visual indexing

Добавить:

- scene detection;
- thumbnails на ключевых сценах;
- contact sheets;
- оценку технического качества;
- сохранение точных source timecodes;
- возможность агента ссылаться на assetId и sourceIn/sourceOut .

Блок D. Audio

Добавить:

- извлечение речи;
- Whisper word timestamps;
- тишину и длинные паузы;
- громкость/LUFS;
- BPM и beat map;
- музыкальные секции и downbeats;

- нормализацию финального mix.

Блок E. Timeline engine

Если ваш текущий рендер-компонент жёстко собирает split/fullscreen, нужен универсальный renderer из [edit_plan.json](#) :

- видео и фото items;
- trim и speed;
- crop/cover/contain;
- Ken Burns для фото;
- transitions;
- overlays;
- captions;
- CTA cards;
- price cards;
- before/after split;
- B-roll поверх talking-head;
- background music + ducking;
- SFX.

Блок F. Brand presets

Вынести из типов рендера в отдельный конфиг:

- цвета;
- шрифты;
- caption position;
- hook typography;
- transition set;
- CTA style;
- logo/watermark;
- safe zones.

Все параметры должны загружаться из [brands/<brand>/visual.json](#) .

Блок G. QA

Автоматические проверки:

- schema validation;
- missing files;
- zero-duration timeline items;
- timeline overlaps/gaps;
- text overflow;
- missing fonts;
- black frames;
- silent audio;
- output metadata;
- render smoke test первых/средних/последних кадров;
- наличие СТА для коммерческого preset.

Блок H. Versioning

Каждая правка создаёт:

```
edit_plan.v1.json  
reel.v1.mp4  
edit_plan.v2.json  
reel.v2.mp4
```

Никаких безымянных final-final-2.mp4 .

38

38. Первая тестовая задача для нового pipeline

Не начинать с хаотичной папки из сотен файлов. Первый end-to-end тест:

Исходники

- одно исходное фото заказчика;
- 3–5 коротких видео процесса;
- 2–3 фото промежуточных этапов;
- 2 видео готовой работы;
- один ваш вертикальный фрагмент или voice-over;
- один разрешённый музыкальный трек;
- подтверждённая цена/срок;
- один СТА.

Цель

25–35-секундный Reel:

Хук → исходное фото → процесс → детали → готовая работа
→ цена/условие → «ЗАКАЗ»

Варианты

- v1 : понятный коммерческий;
- v2 : эмоциональный подарок;
- v3 : быстрый beat-sync.

Успешный тест

- все три версии собираются из одного manifest;
- замена хука не требует ручной переклейки;
- цена берётся из брендового источника;
- при замене трека beat map перестраивается;
- вы можете попросить одну правку обычным языком;
- новая версия сохраняется отдельно;
- готовый Reel проходит QA.

39. Практические команды по дням

Промпты ниже написаны от первого лица — вводите их как есть в Codex CLI/ChatGPT Desktop или в Claude Code CLI/Claude Desktop, смотря что у вас выбрано. Менять в них ничего не нужно: команды сформулированы так, чтобы работать в обоих инструментах, а упоминания `$reel`, `$talking-head` и остальных skills одинаковы и там, и там (разделы 31–32).

День 1

```
Проверь окружение этого проекта: сам агент, FFmpeg, ffprobe, Node,
npm и Python. Ничего не устанавливай без моего подтверждения.
Покажи таблицу: компонент, версия, статус, что нужно исправить.
```

День 2

```
Используй studio-brand. Проверь новую папку в inbox,
создай manifest, транскрипт, ключевые кадры и contact sheet.
После анализа покажи, какие исходники сильные, слабые и отсутствуют.
```

День 3

```
$reel создай три концепции для первой задачи studio.
Цель — заявки на заказ. Не придумывай цены и не рендери,
пока я не выберу концепцию.
```

День 4

\$talking-head подготовь три сценария до 45 секунд по теме «как выбрать фото для заказа». После выбора сделай версию для суфлёра и список кадров, которые нужно доснять.

День 5

\$repurpose возьми утверждённый Reel и подготовь отдельные версии для Instagram, Telegram, MAX и VK. Сохрани их разными файлами.

День 6

Подготовь спецификацию ChatPlace-воронки по слову ЗАКАЗ: триггер, первое сообщение с кнопкой, вопросы, переменные, теги, ветку ручного подключения и тест-кейсы. Пока не активируй её.

День 7

Проведи полный rehearsal: новая папка → анализ → концепции → render → QA → platform copy → publication drafts. Зафиксируй все места, где потребовалась помощь, и обнови пошаговую инструкцию.

40. Итог семи рабочих дней

Неделя считается завершённой не по количеству пройденных шагов, а по работающему результату на ваших материалах.

Нужны четыре доказательства:

1. **Доказательство монтажа:** готовый Reel, созданный новым pipeline из ваших исходников.
2. **Доказательство повторяемости:** второй Reel собирается тем же skill без ручного переписывания кода.
3. **Доказательство самостоятельности:** вы сами запускаете команду, просите правку и находите результат.
4. **Доказательство бизнеса:** тестовое кодовое слово проходит воронку и создаёт заявку, которую можно обработать человеком.

Если нет хотя бы одного доказательства, его надо вынести в незавершённый backlog, а не обозначать как «всё настроено».
